Last Lecture

Eugene Wu

Agenda

1. Project Winners
2. Modern Databases
3. Q&A / Review

Grading

Normalize all assignments to [0-1]

Weighed sum based on assignment weights

Compute cut-offs, ~ B+ avg

Then add in extra credit

Project 1 Part 3 Winners

xs2485	hl3608
dk3245	hr2538
sk4819	gj2353
afl2124	hz2771
sks2243	jfu2001
hm2953	msb2252

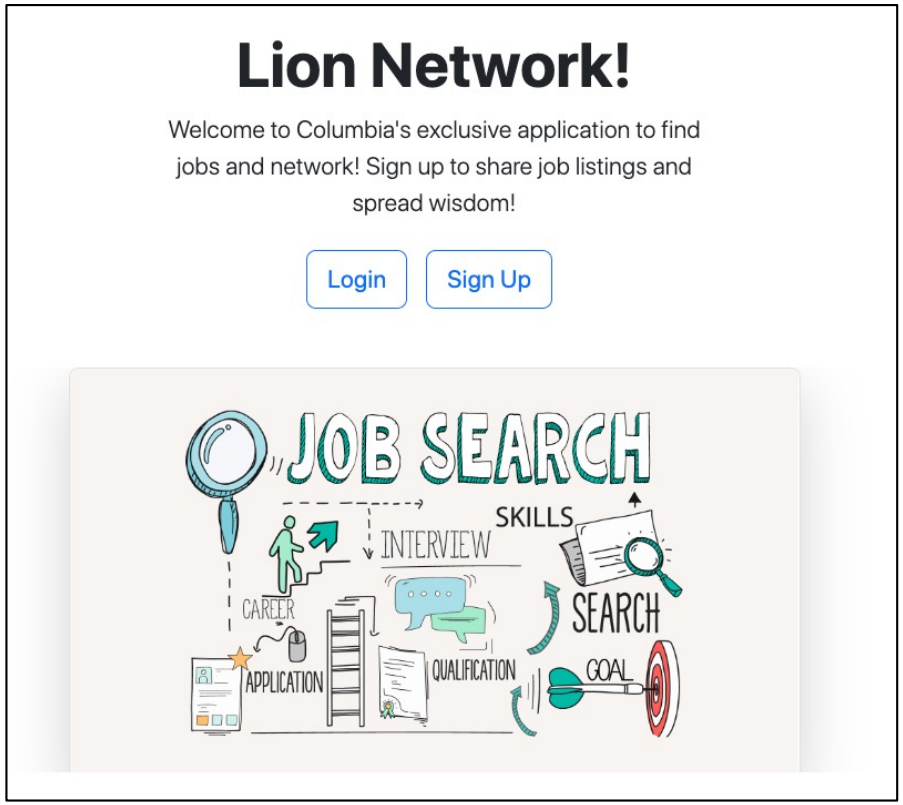
Food

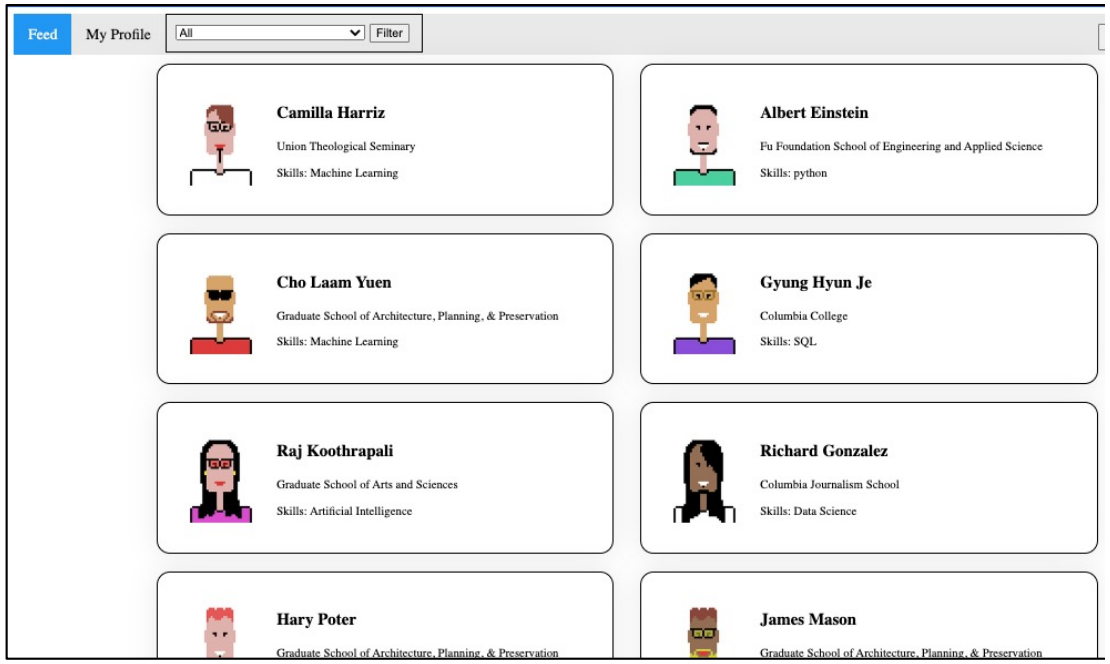
Welcome to NYC restaurant inspection & rating database

Welcome to log in to use our databse! Our database help you find awesome restaurants in NYC! You can check restaurants' information, ratings, sanity inspection data and violations at our database, which helps you build you favourite restaurant list as well as a hated list for restaurants in NYC. Share you thoughts with other users and explore other's fav restaurant. Hope you enjoy our database!

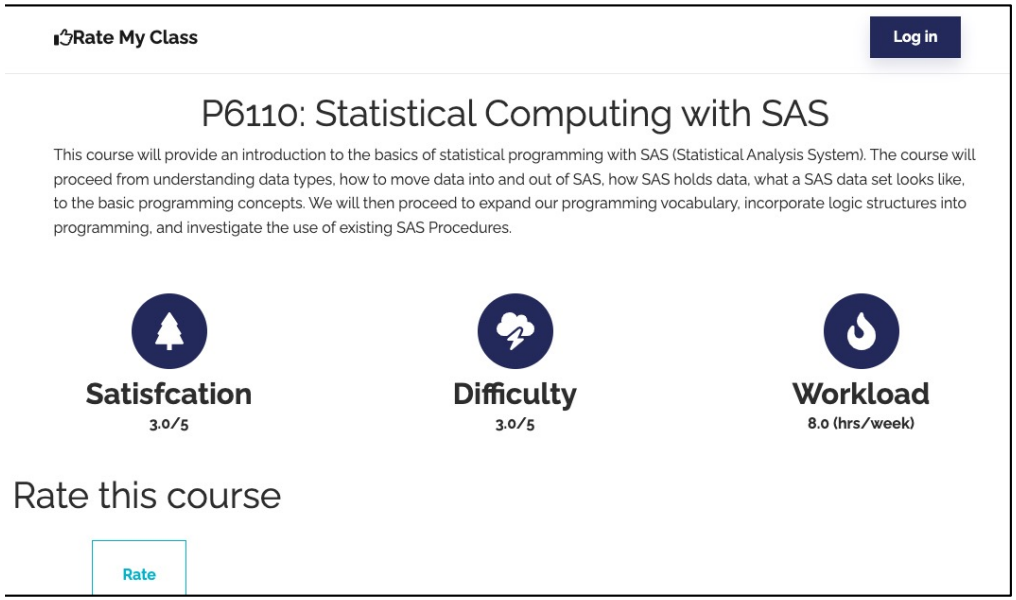
Log in / Sign up

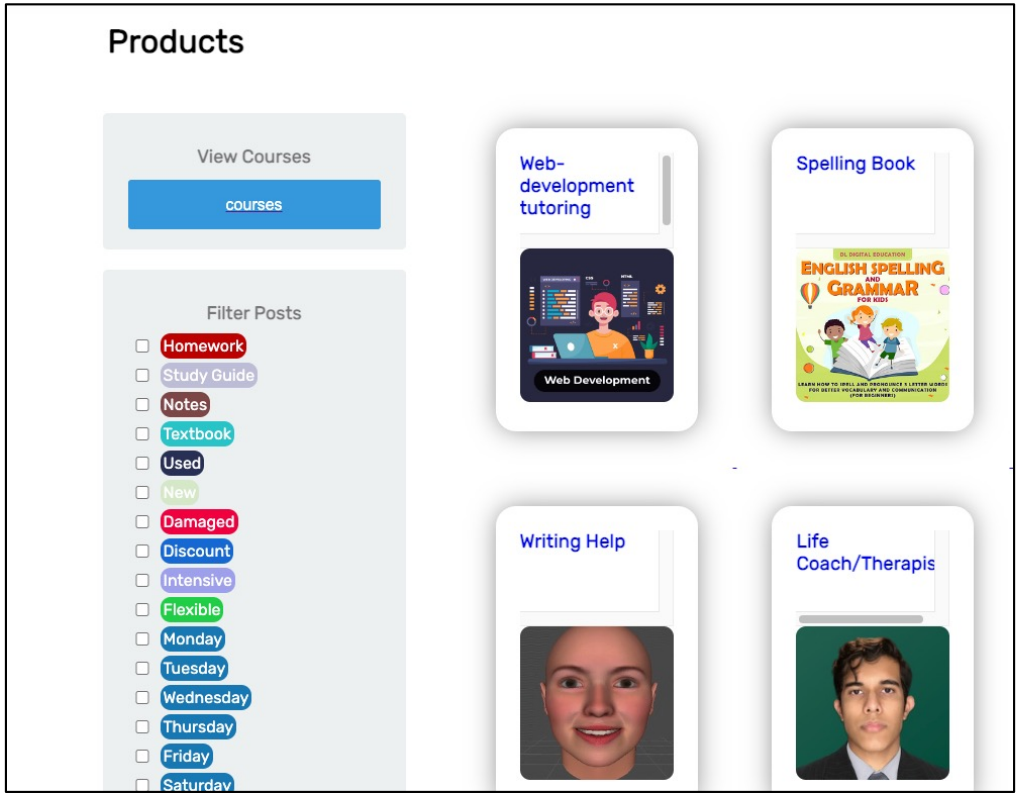
Jobs


Lion Network!
Welcome to Columbia's exclusive application to find jobs and network! Sign up to share job listings and spread wisdom!
Login
Sign Up
JOB SEARCH
SKILLS
INTERVIEW
CAREER
SEARCH
APPLICATION
QUALIFICATION
GOAL

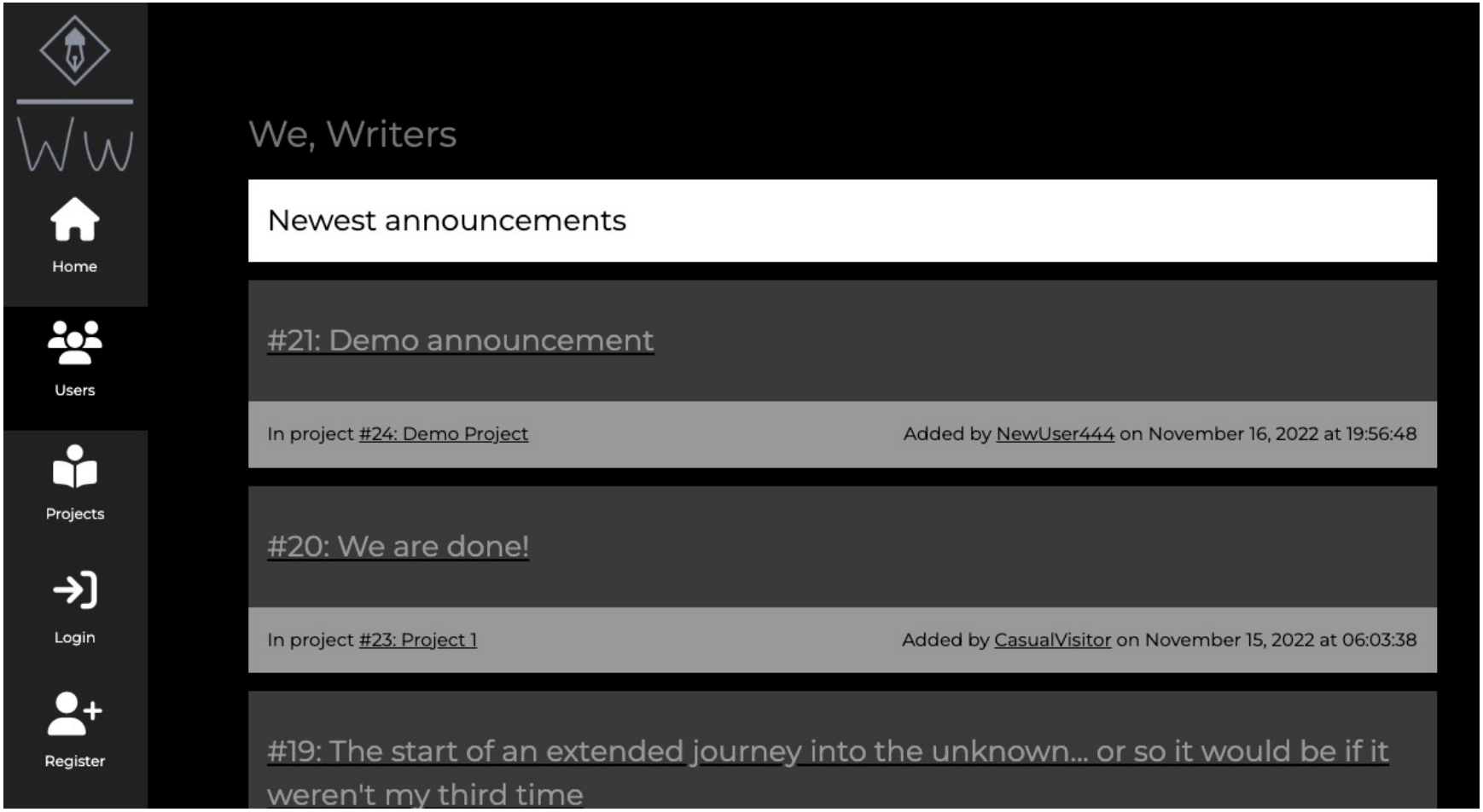

Feed
My Profile
All
Filter
Camilla Harriz
Union Theological Seminary
Skills: Machine Learning
Albert Einstein
Fu Foundation School of Engineering and Applied Science
Skills: python
Cho Laam Yuen
Graduate School of Architecture, Planning, & Preservation
Skills: Machine Learning
Gyung Hyun Je
Columbia College
Skills: SQL
Raj Koothrapali
Graduate School of Arts and Sciences
Skills: Artificial Intelligence
Richard Gonzalez
Columbia Journalism School
Skills: Data Science
Hary Poter
Graduate School of Architecture, Planning, & Preservation
James Mason
Graduate School of Architecture, Planning, & Preservation

Courses


Rate My Class
Log in
P6110: Statistical Computing with SAS
This course will provide an introduction to the basics of statistical programming with SAS (Statistical Analysis System). The course will proceed from understanding data types, how to move data into and out of SAS, how SAS holds data, what a SAS data set looks like, to the basic programming concepts. We will then proceed to expand our programming vocabulary, incorporate logic structures into programming, and investigate the use of existing SAS Procedures.
Satisfcation
3.0/5
Difficulty
3.0/5
Workload
8.0 (hrs/week)
Rate this course
Rate


Products
View Courses
courses
Filter Posts
Homework
Study Guide
Notes
Textbook
Used
New
Damaged
Discount
Intensive
Flexible
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Web-development tutoring
Web Development
Spelling Book
ENGLISH SPELLING AND GRAMMAR
Writing Help
Life Coach/Therapis

Writing Community



DBMSes in the Wild

Classic Relational

- $$: Oracle, IBM, Microsoft, Teradata, EMC, etc
- Free: MySQL, PostgreSQL

New Relational

- In-Memory, Column-store, Streaming

Non-traditional

- Search (Google, Bing, Lucene), Scientific, Geographic

NoSQL

- Big Data: Hadoop, Spark, etc
- Key-value: Mongo, BerkeleyDB, Cassandra, etc

DBMS-as-a-Service

- Microsoft Azure, Amazon Redshift/RDS, etc…

DBMSes in the Wild

Classic Relational

- $$: Oracle, IBM, Microsoft, Teradata, EMC, etc
- Free: MySQL, PostgreSQL

New Relational

- In-Memory, Column-store, Streaming

Non-traditional

- Search (Google, Bing, Lucene), Scientific, Geographic

NoSQL

- Big Data: Hadoop, Spark, etc
- Key-value: Mongo, BerkeleyDB, Cassandra, etc

DBMS-as-a-Service

- Microsoft Azure, Amazon Redshift/RDS, etc…

Modern Database Systems

Hardware changes affect

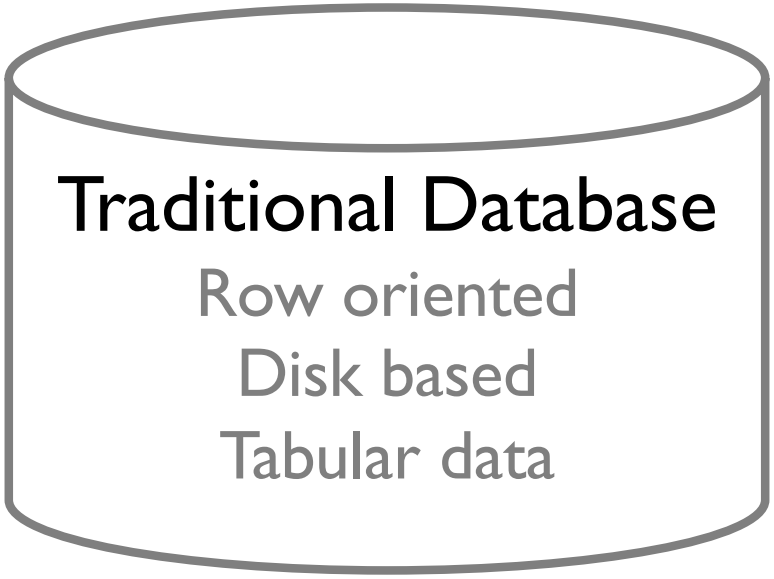
Compression is good → Column stores

- Large scale aggregation queries
- "Data Warehouses"

Memory is cheap → In-memory stores

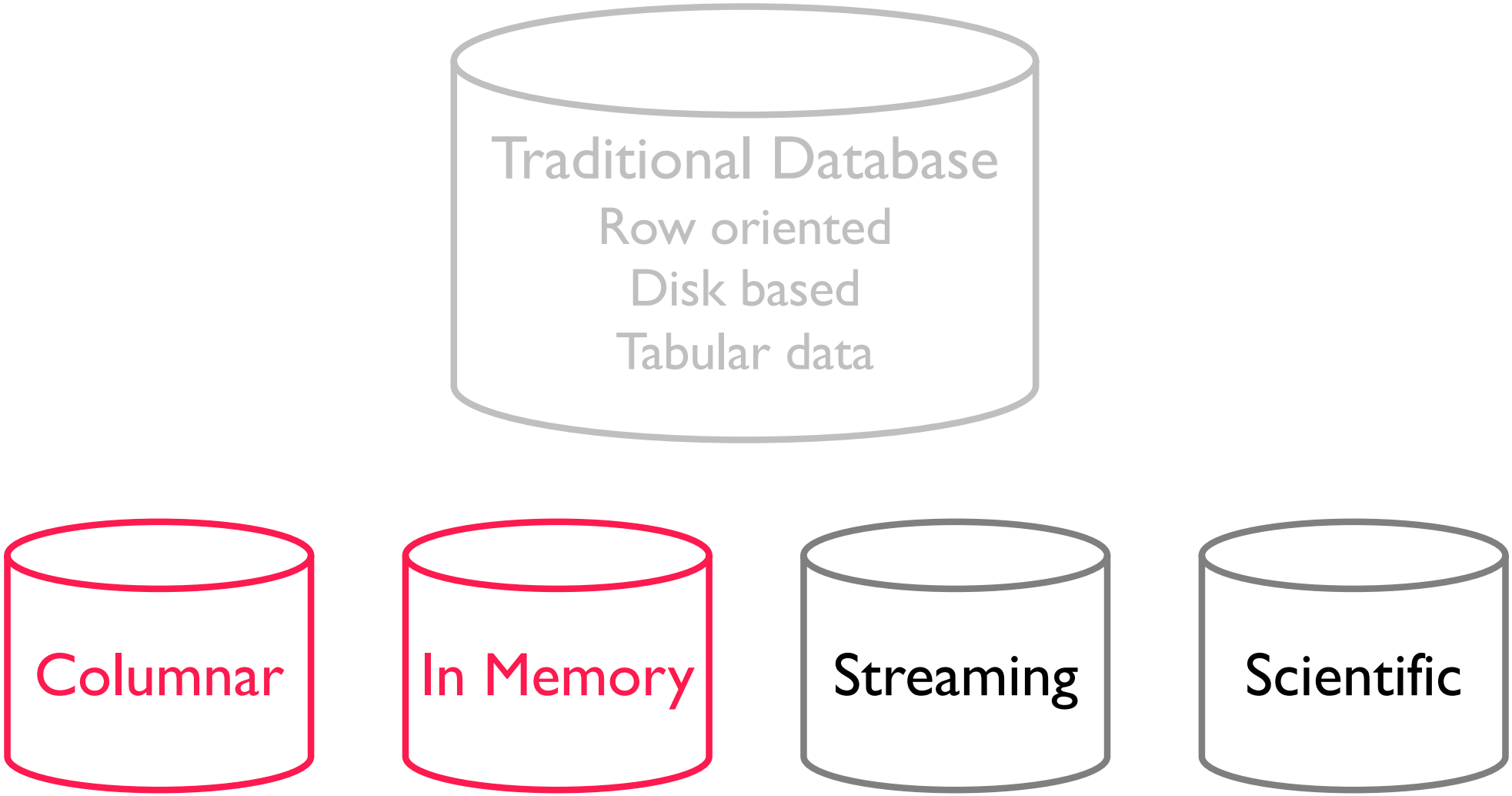
- Transactional systems

One Size Fits All



One Size Does Not Fits All

One Size Does Not Fits All



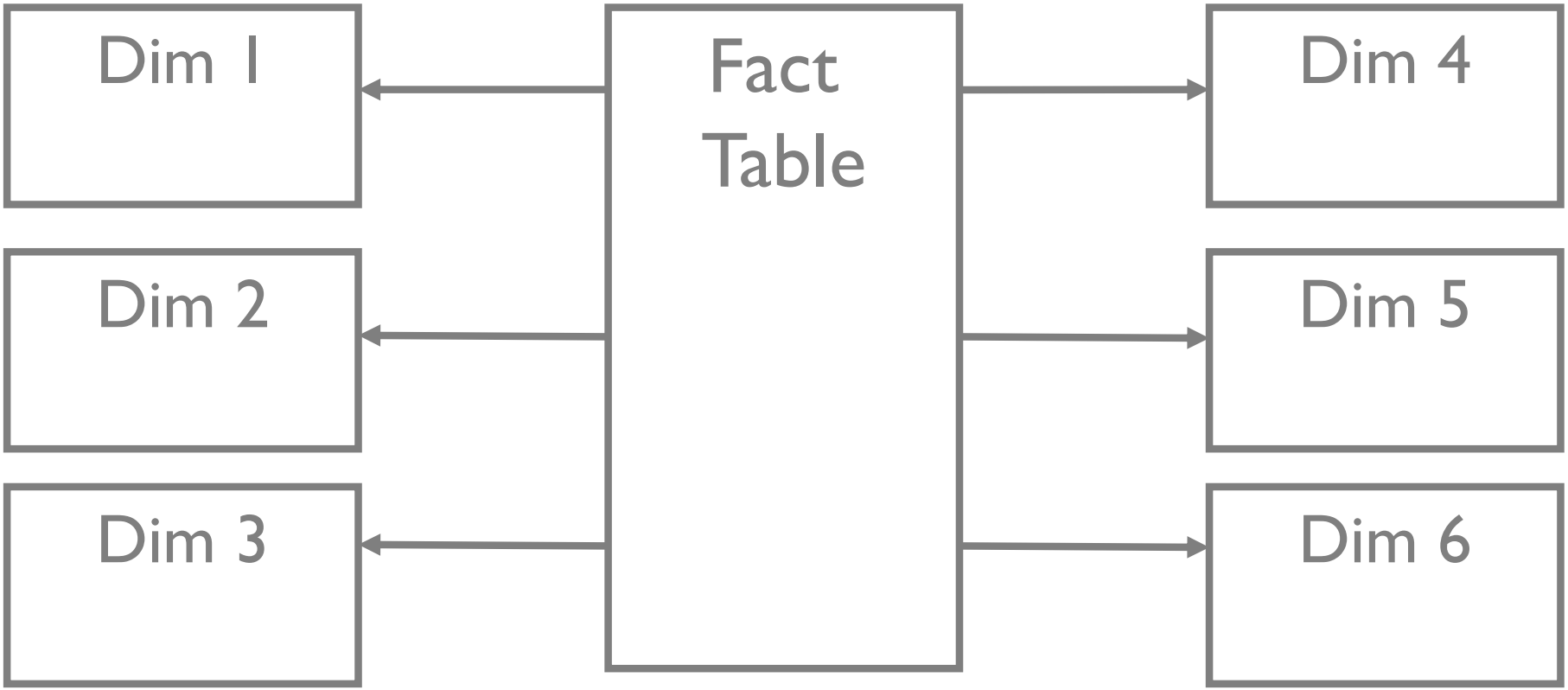
Data Warehouses

Store all historical data for future analysis

- Sales by month over past 20 years
- Clicks by youth in texas
- Cost by product component

Defacto standard at any company

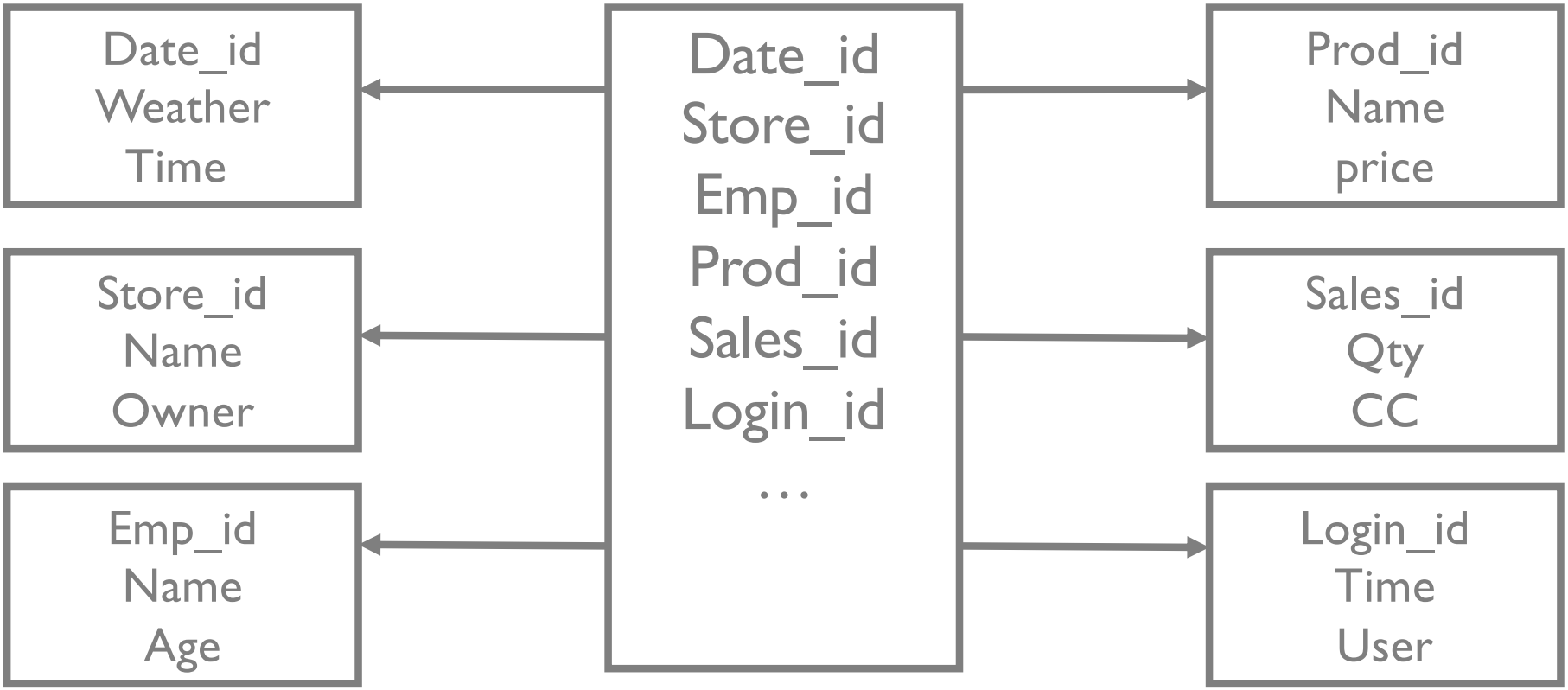
Star Schema



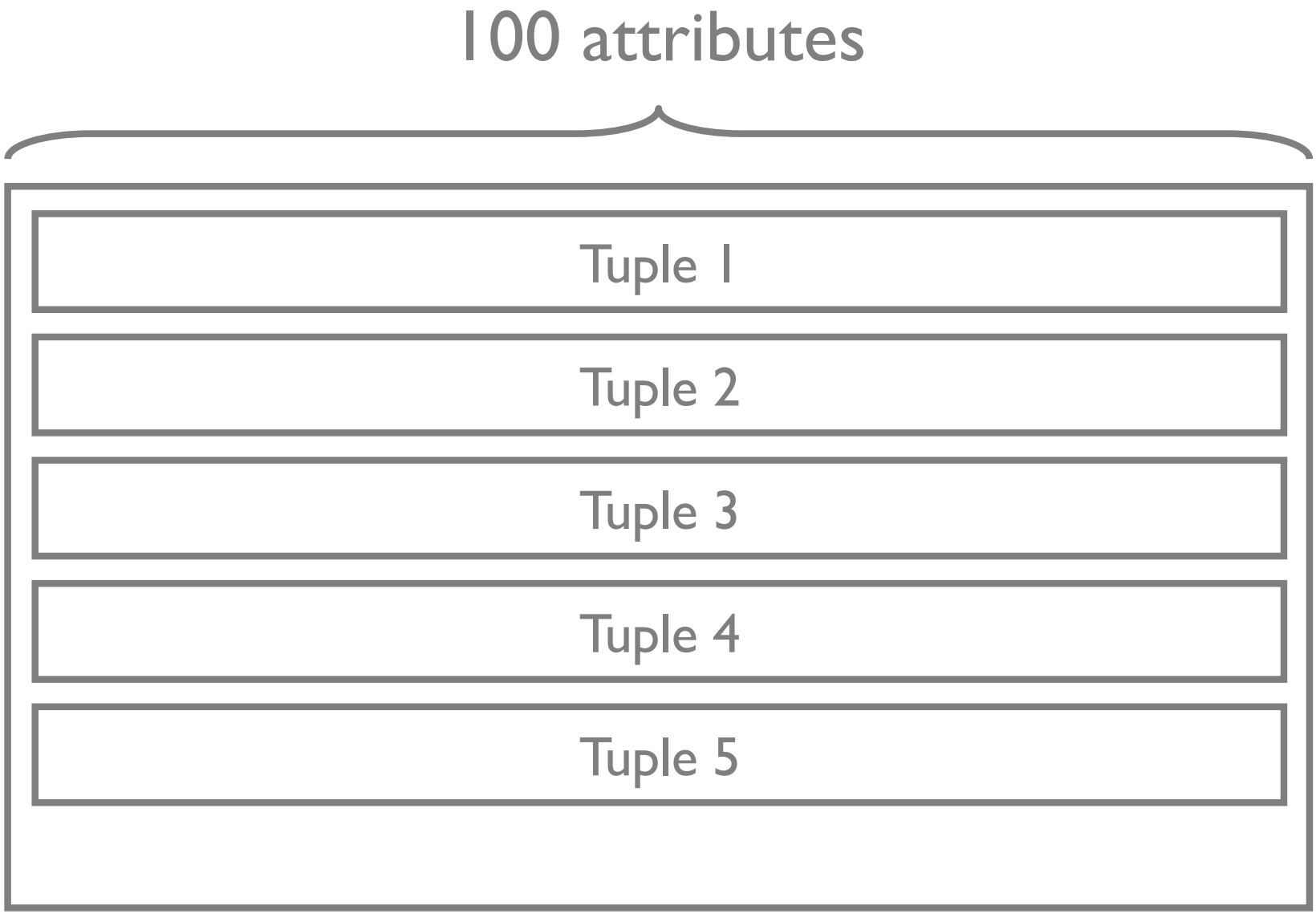
Star Schema

Fact table is "fat"

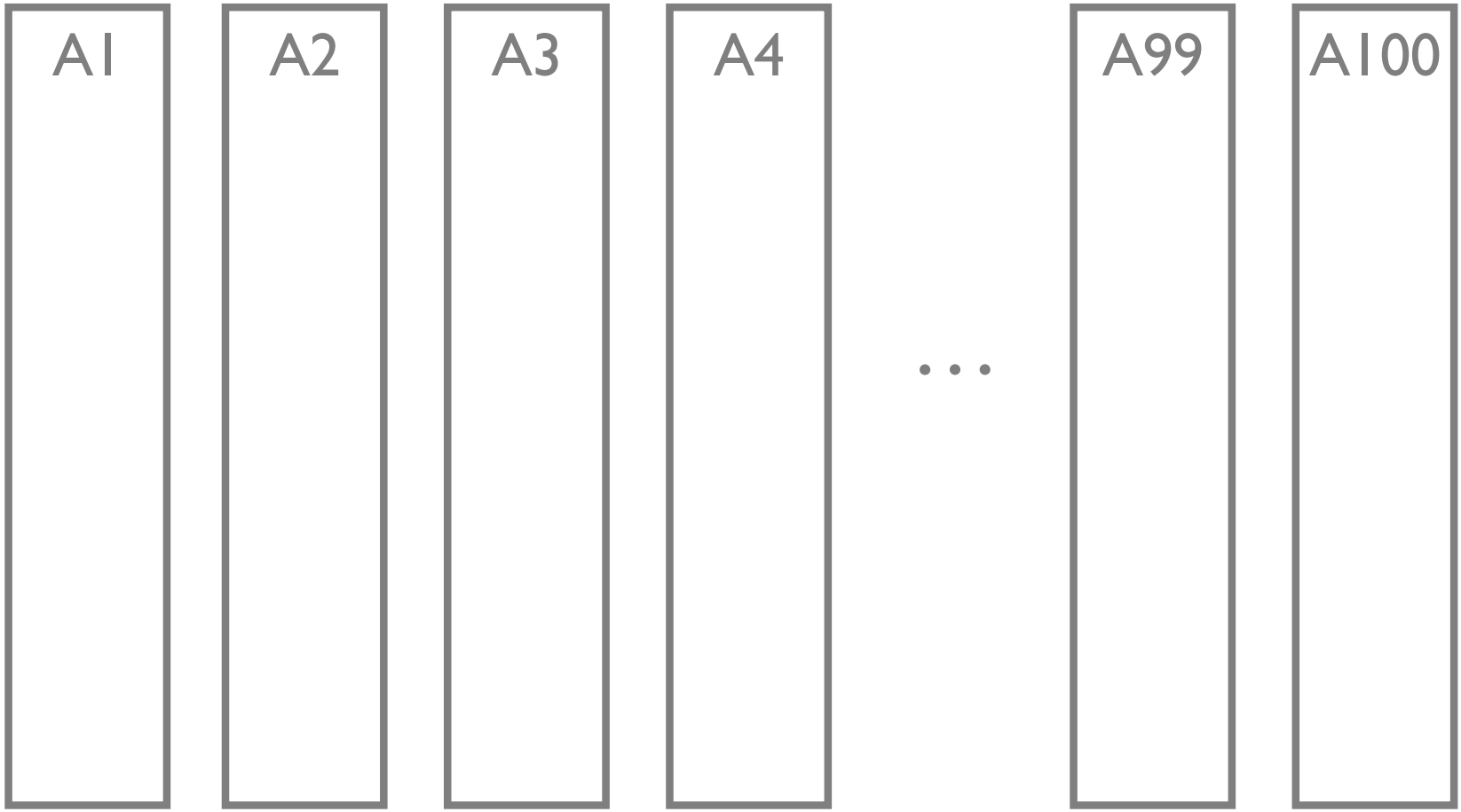
Queries access ~6 attrs



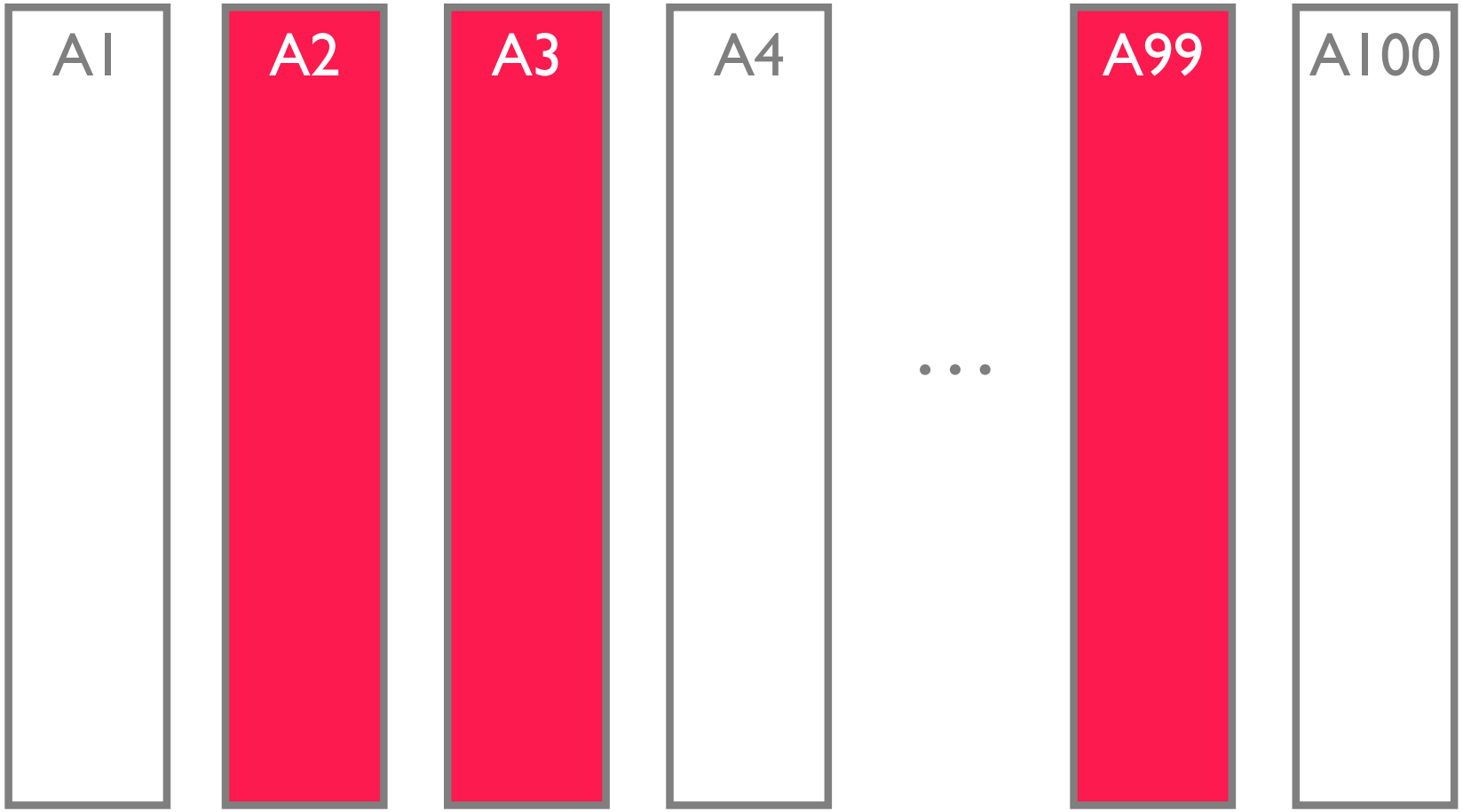
```
SELECT A2, A3, AVG(A99) GROUP BY A2, A3
```



```
SELECT A2, A3, AVG(A99) GROUP BY A2, A3
```



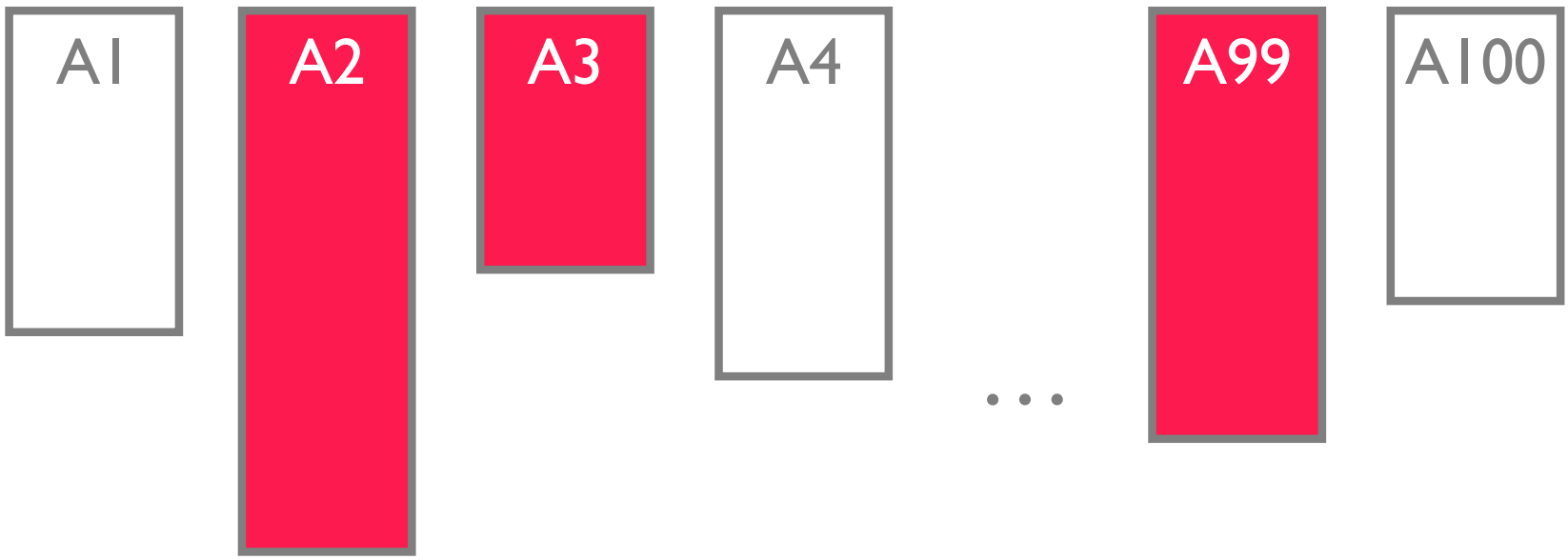
16x less data read. Unfair advantage.



16x less data read. Unfair advantage.

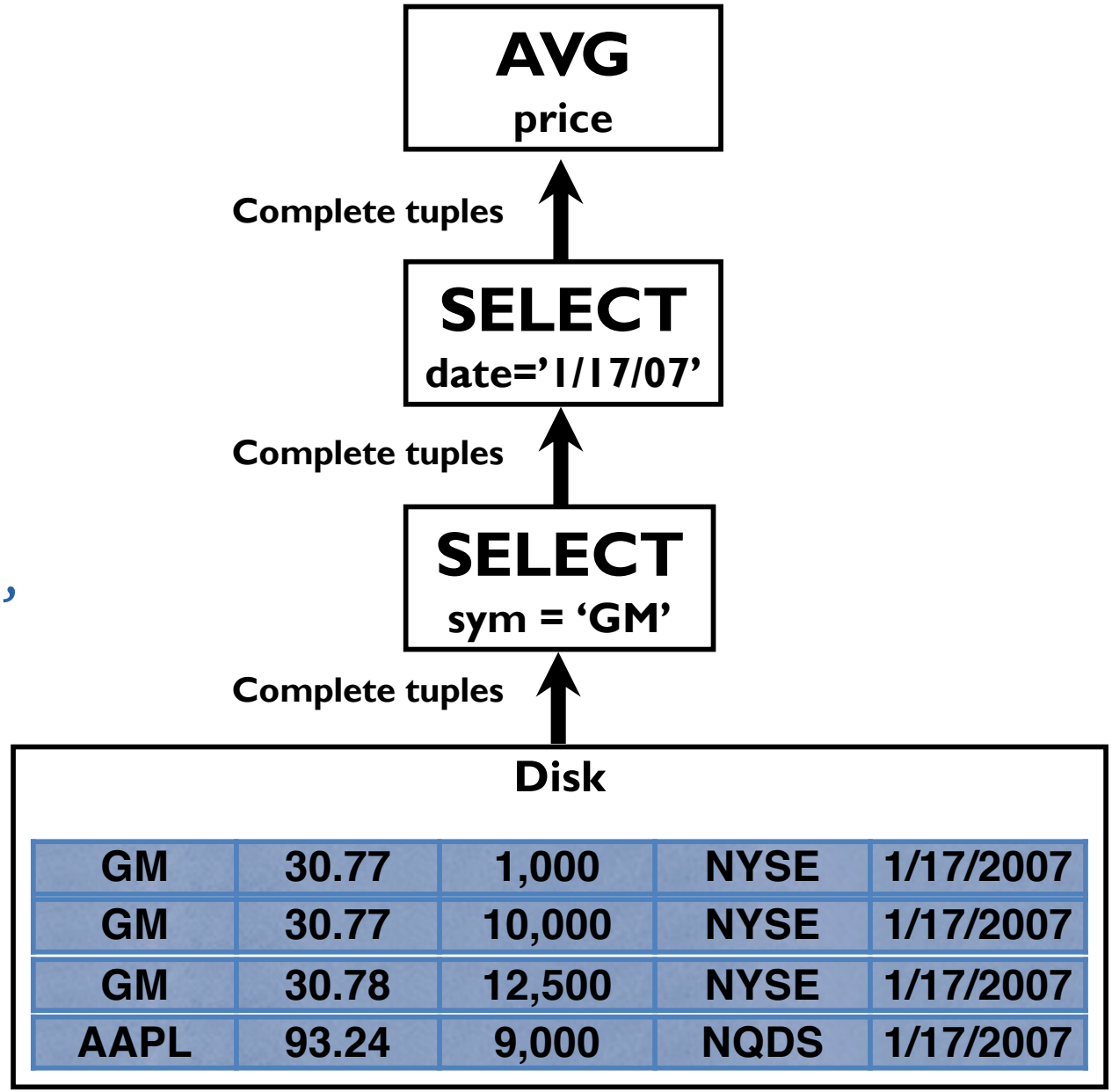
Compression better on single column

Execute on compressed data



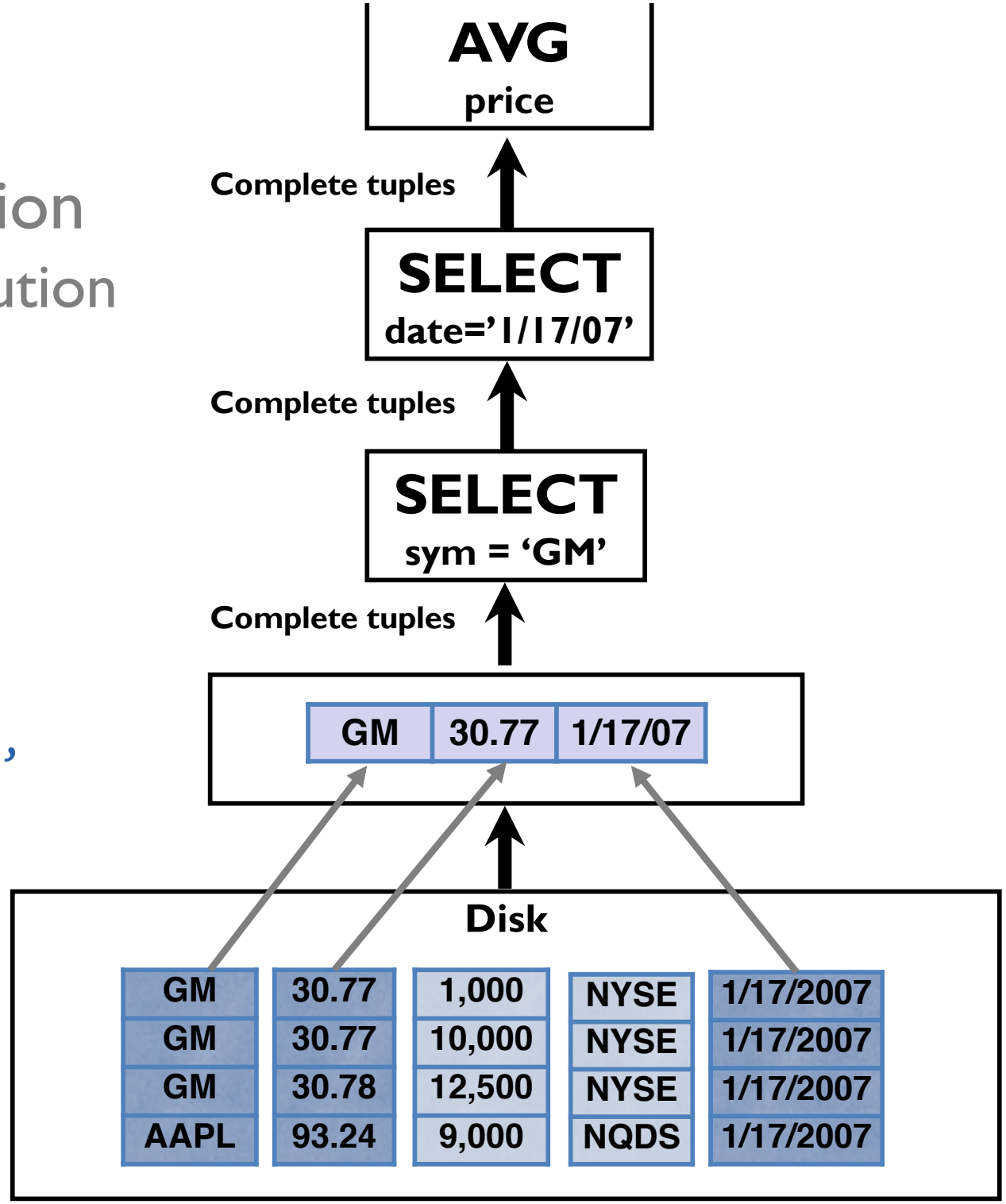
Traditional DBMS

```
SELECT avg(price)
FROM tickstore
WHERE symbol = 'GM'
AND date = '1/17/2007'
```



Naïve:
Early Materialization
Row oriented execution

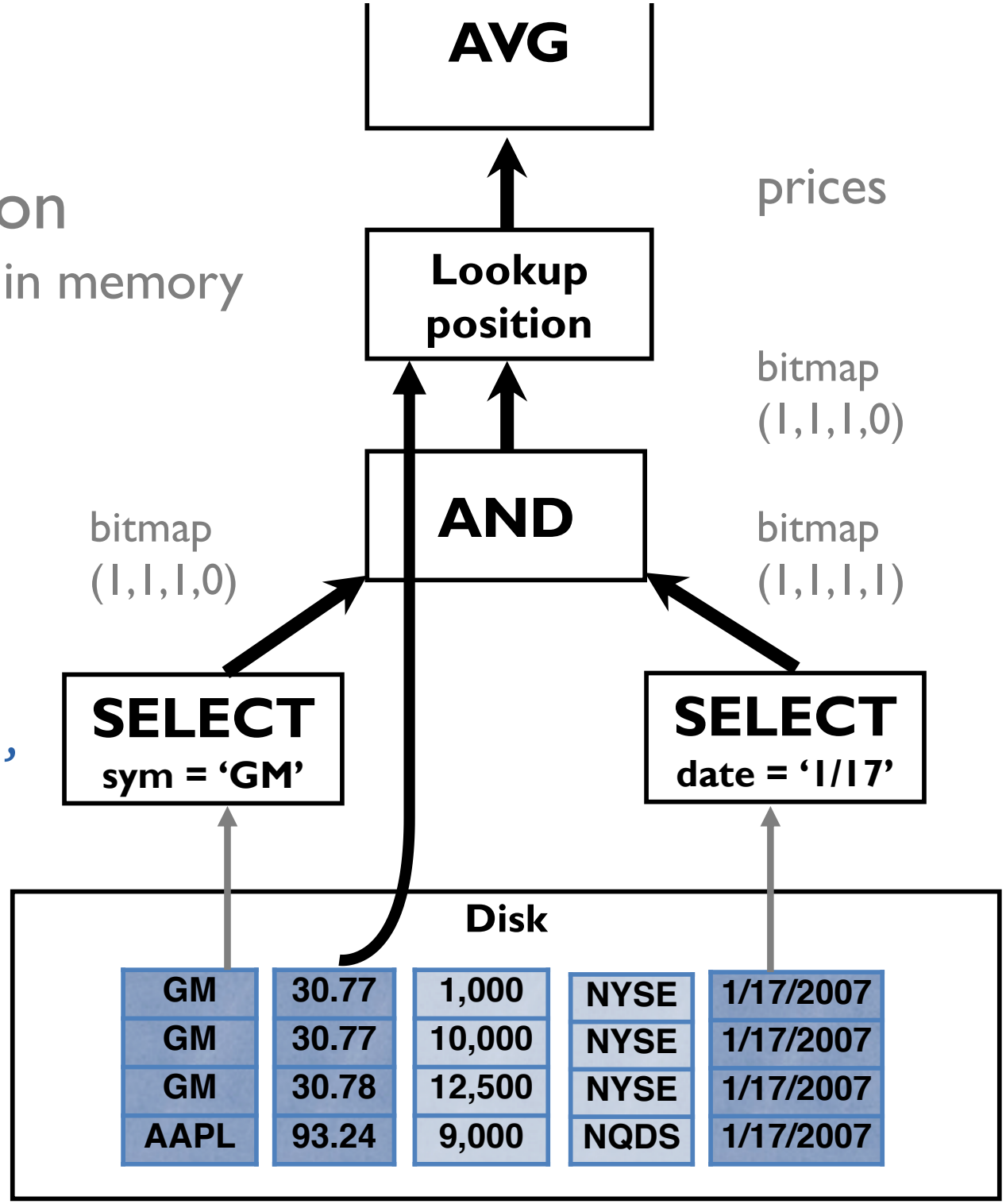
```
SELECT avg(price)
FROM tickstore
WHERE symbol = 'GM'
AND date = '1/17/2007'
```



C-Store
Late Materialization

Much less data moving in memory

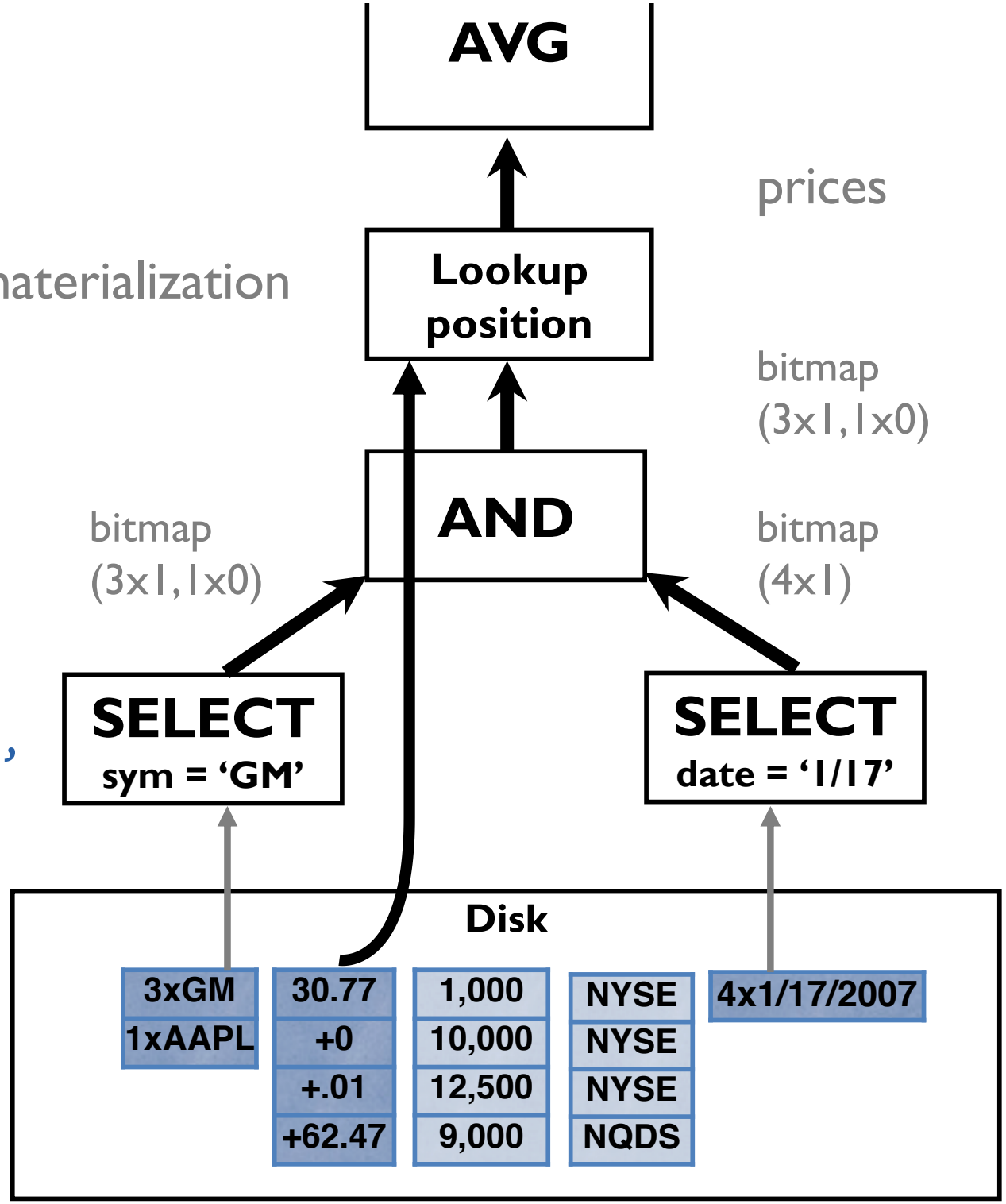
```
SELECT avg(price)
FROM tickstore
WHERE symbol = 'GM'
AND date = '1/17/2007'
```



C-Store Compression

Only possible w/ late materialization

```
SELECT avg(price)
FROM tickstore
WHERE symbol = 'GM'
AND date = '1/17/2007'
```



Column Stores

Optimized for data warehouses

Store data by attribute/column rather than row

Compression

Compressed *query plan execution*

50-100x faster than row store

In-Memory DBMSes

Transaction-oriented apps

- remove 1 unit from product
- move 5 units from org 1 to org 2
- (shopping carts, inventory)

Data stored in memory

- Disk only used for check pointing

No concurrency

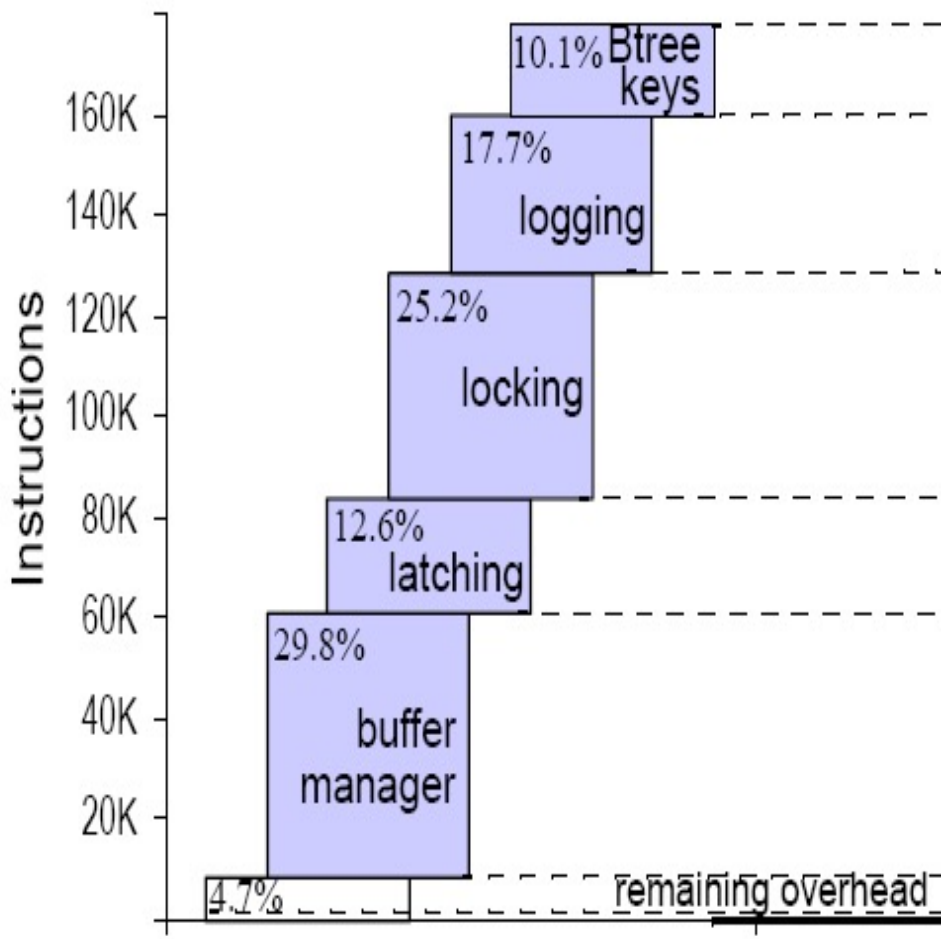
Active-active replication for fault-tolerance

Traditional Database

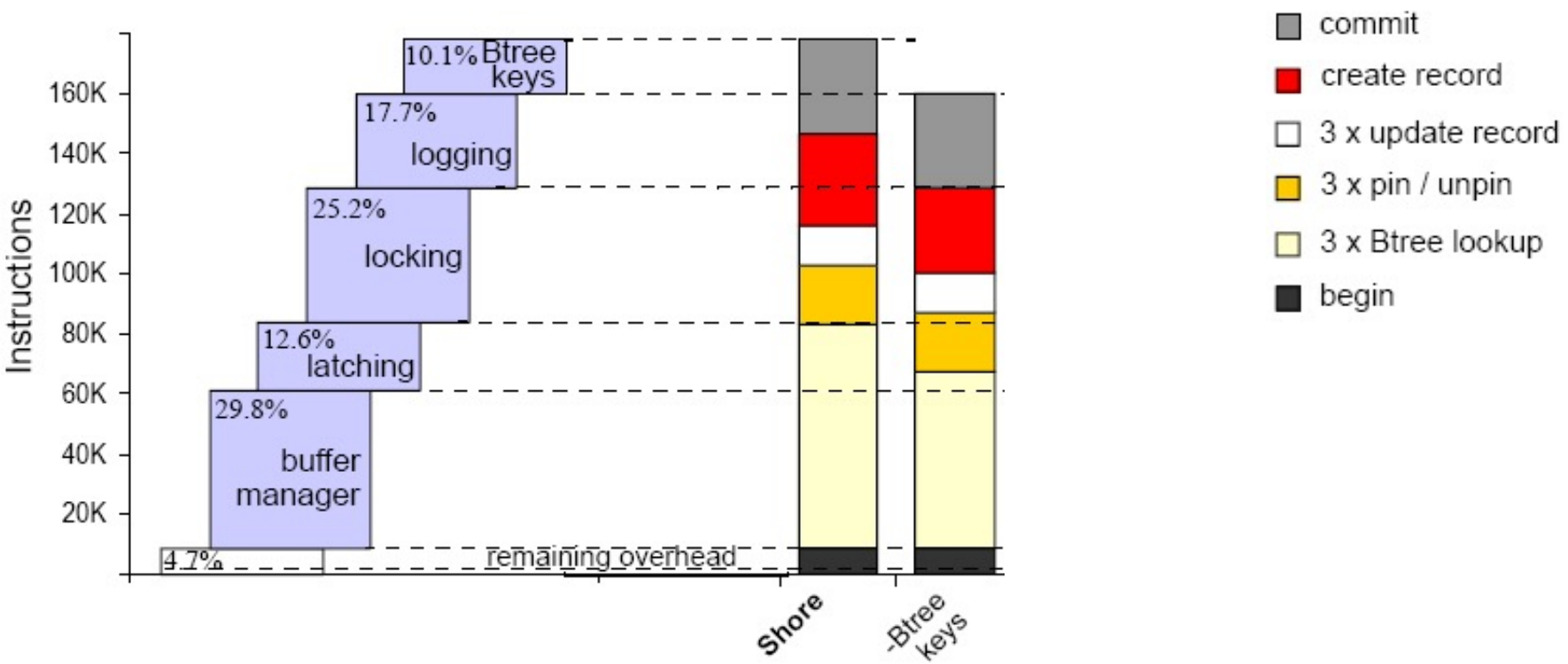
Indexes	queries go faster
Concurrency	queries go faster
Locking	serializability
Logging	recovery
Buffer Manager	manage pages in memory

Results after removing the components (in # instruction)

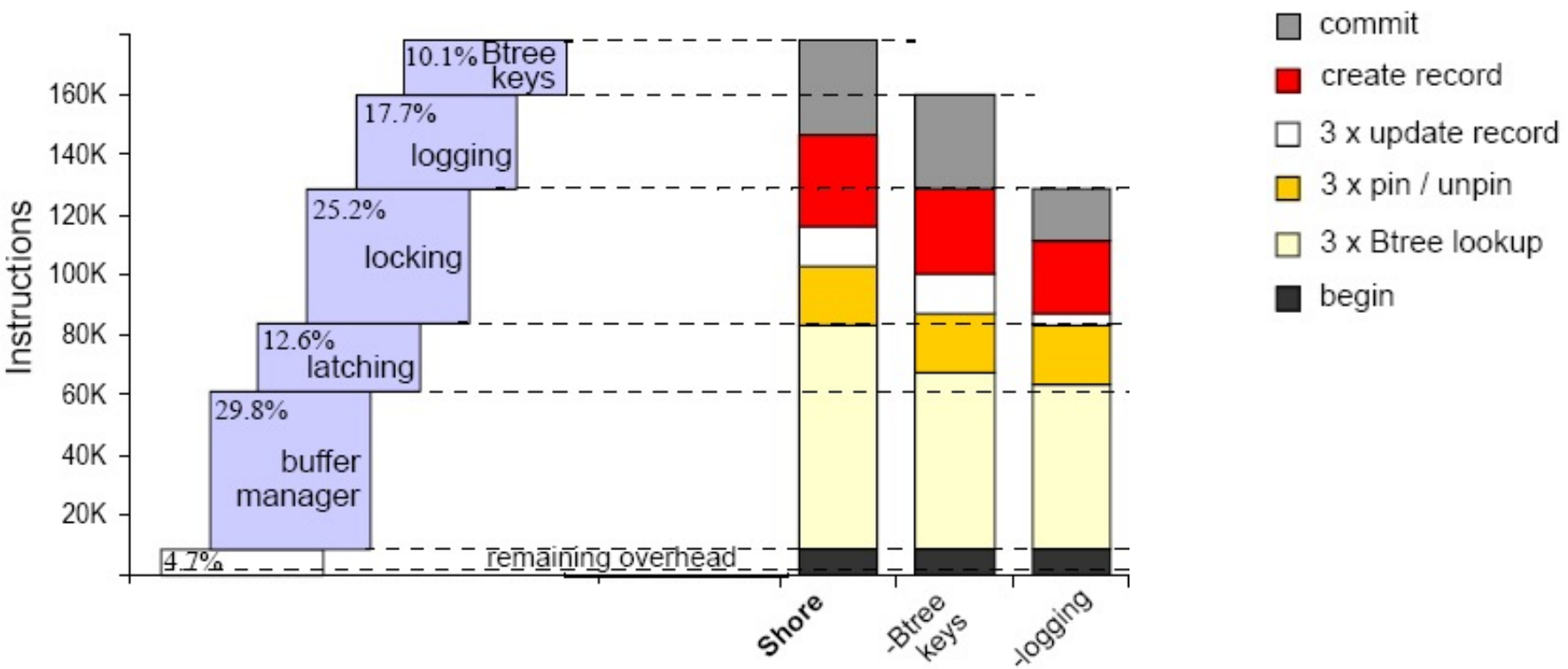
Instruction of useful work is only <2% of a memory resident DB



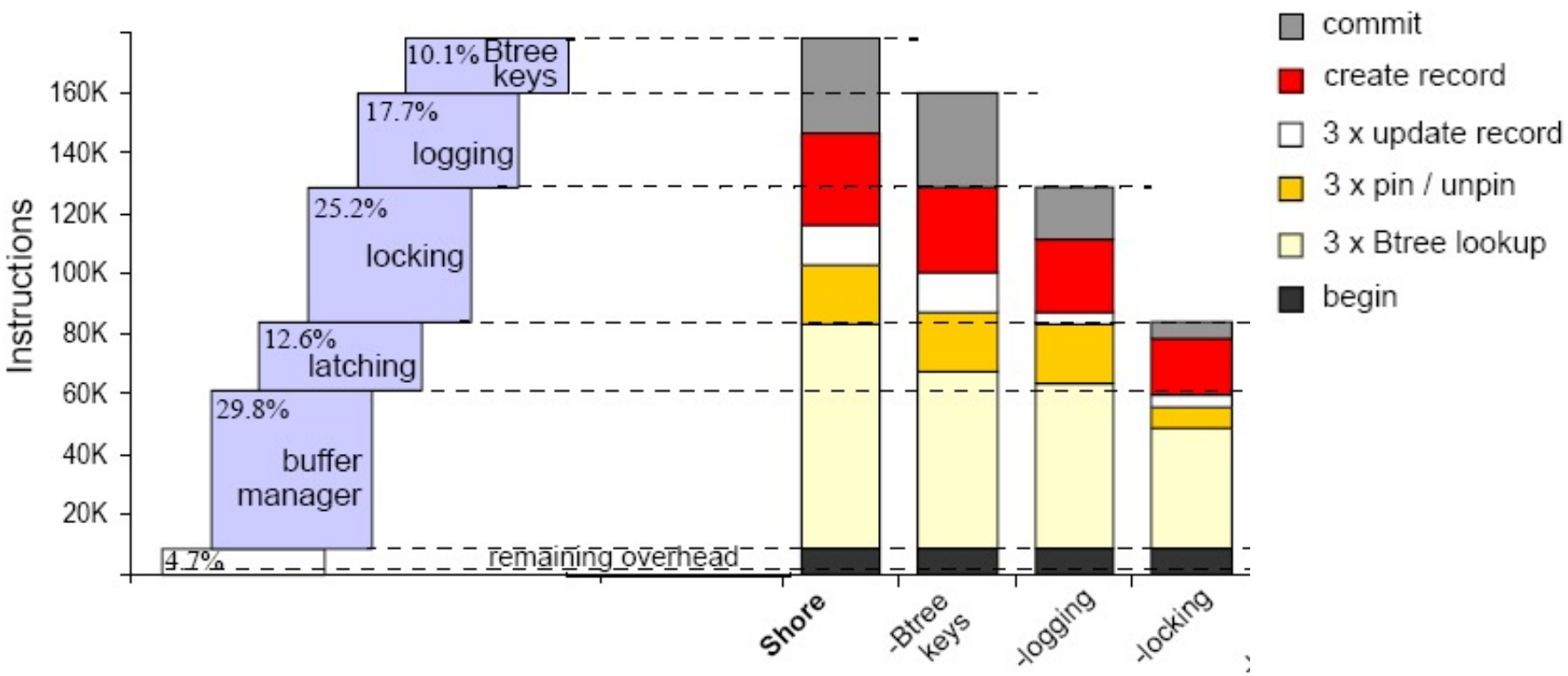
Effect of removing different components for payment (1/6)



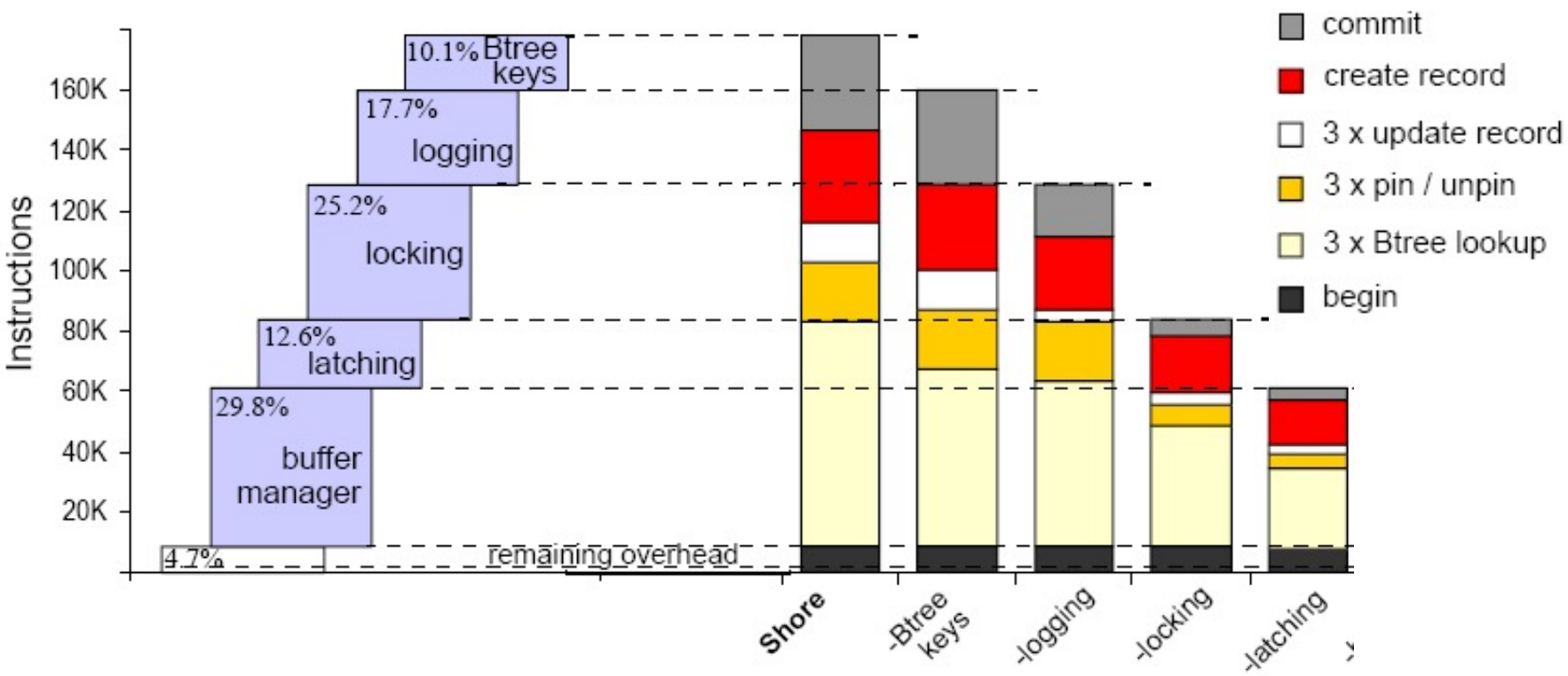
Effect of removing different components for payment (2/6)



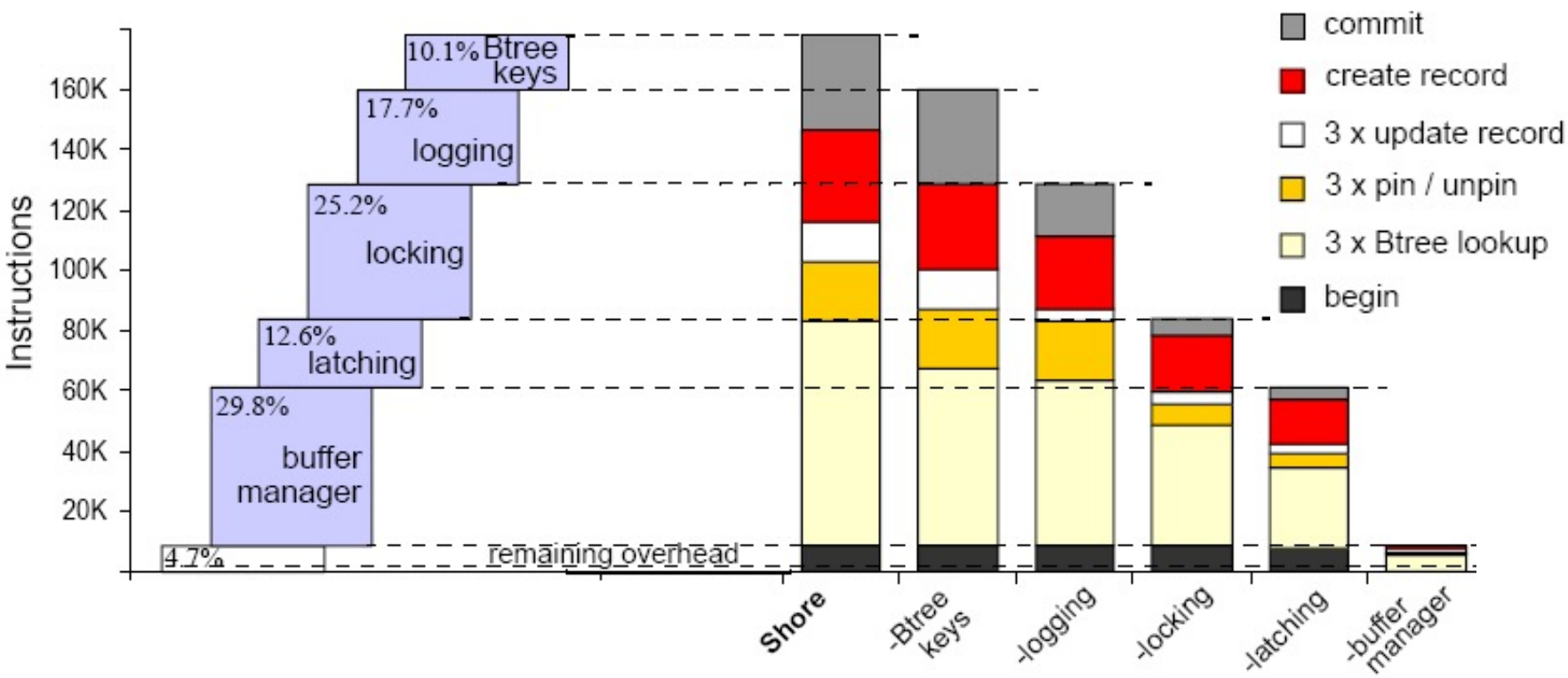
Effect of removing different components for payment (3/6)



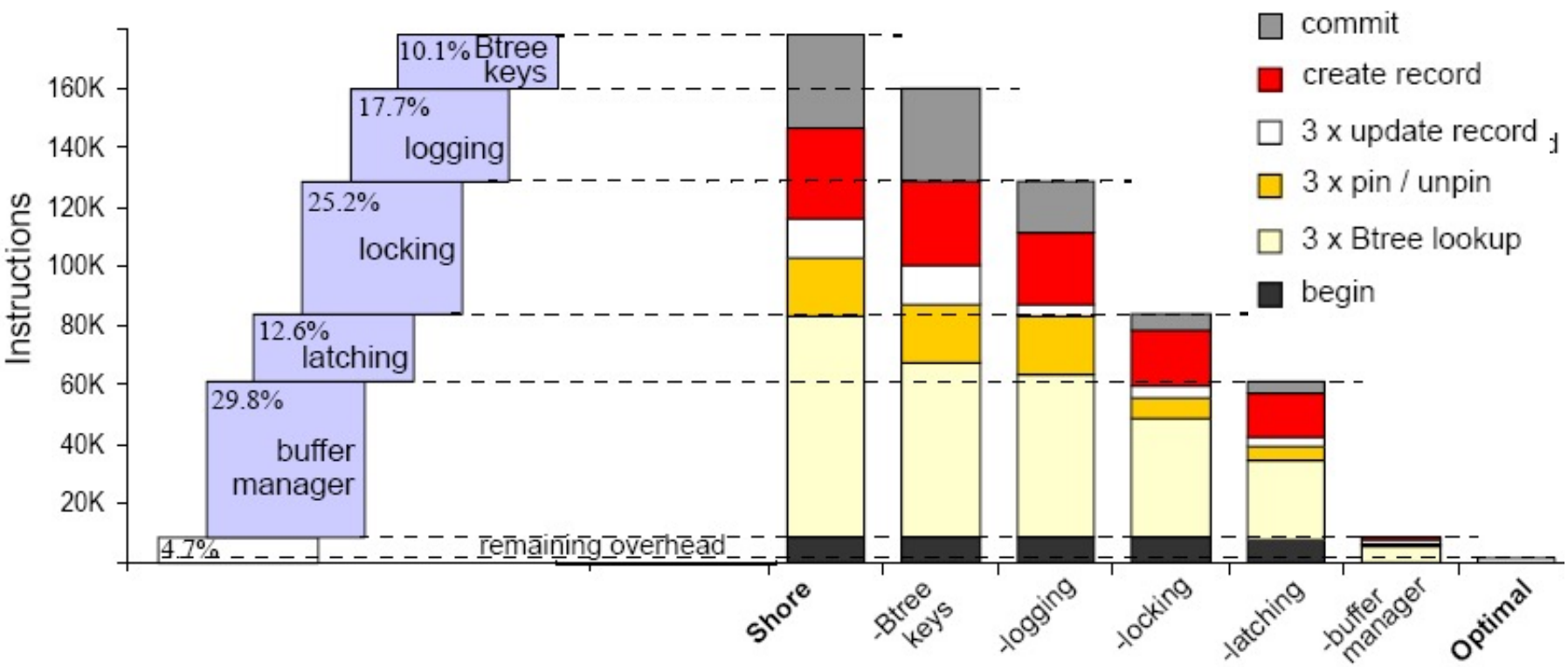
Effect of removing different components for payment (4/6)

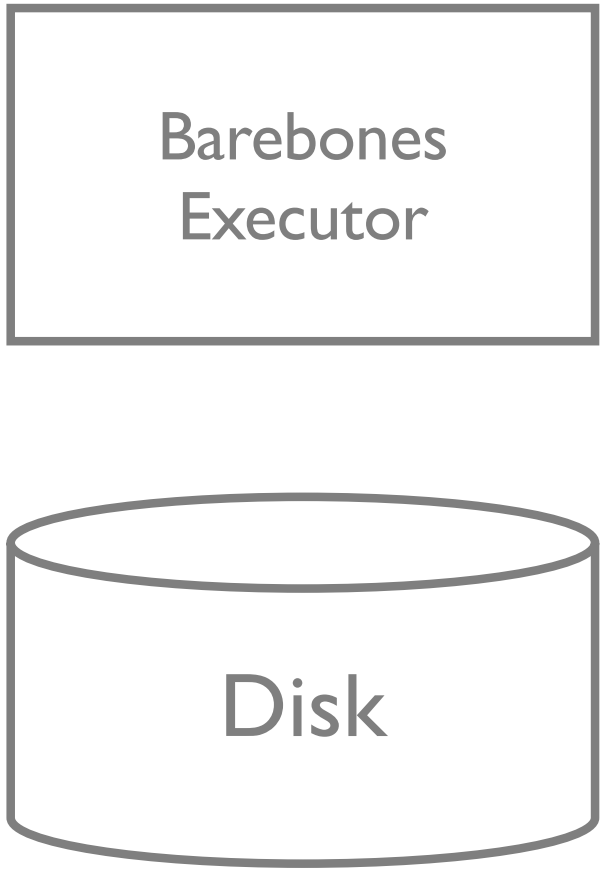


Effect of removing different components for payment (5/6)



Effect of removing different components for payment (6/6)



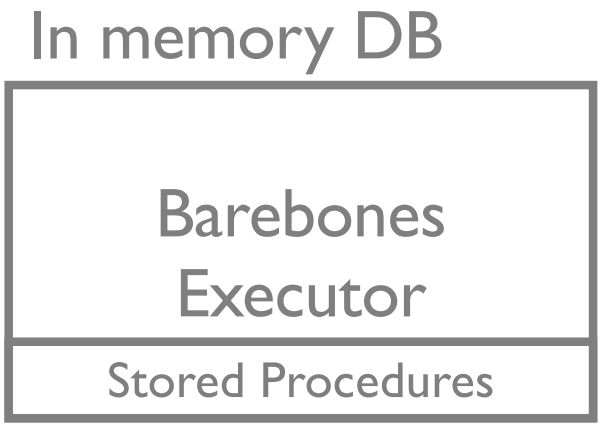

Barebones
Executor
Disk

In memory DB

Barebones
Executor

What about

- Parsing
- Concurrency?
- Recovery?



What about

- Parsing
- Concurrency?
- Recovery?

Procedure:

```
p1 =  SELECT cost
      FROM fact_table
      WHERE id = ?
```

Query:

```
p1(10)
```

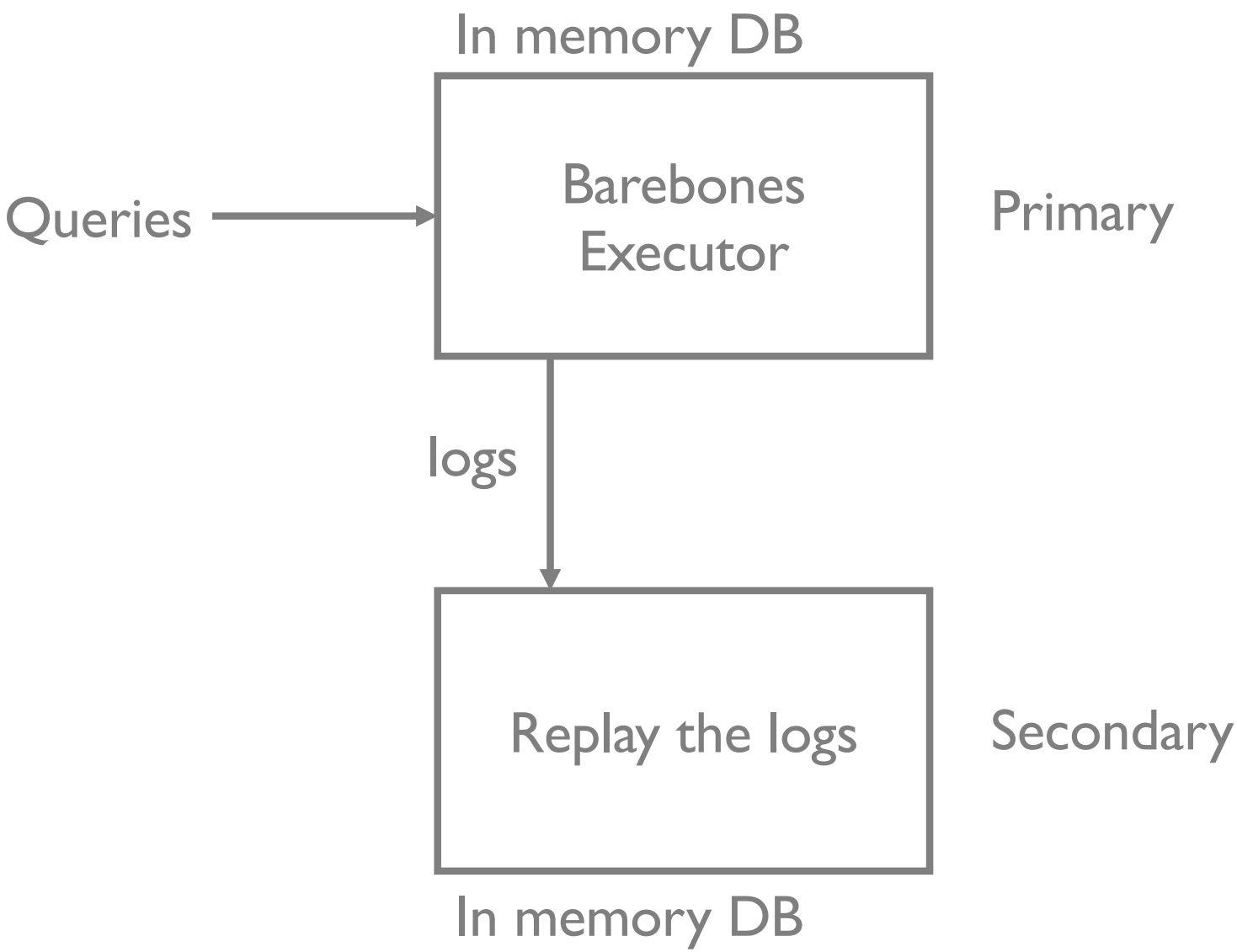
In memory DB

Barebones
Executor

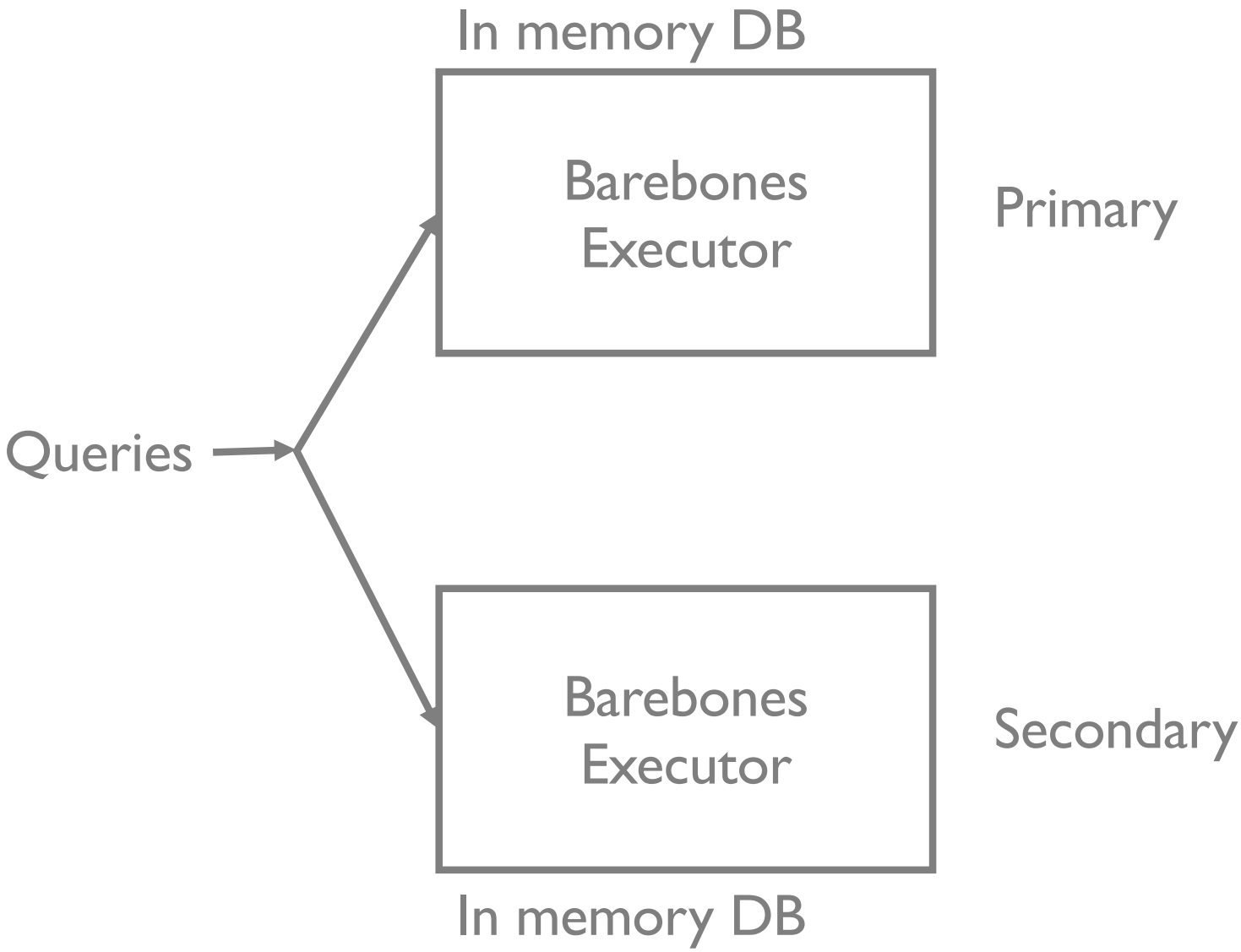
What about

- Parsing
- ~~Concurrency?~~
  - no buffer manager, no concurrency, no locks
- Recovery?

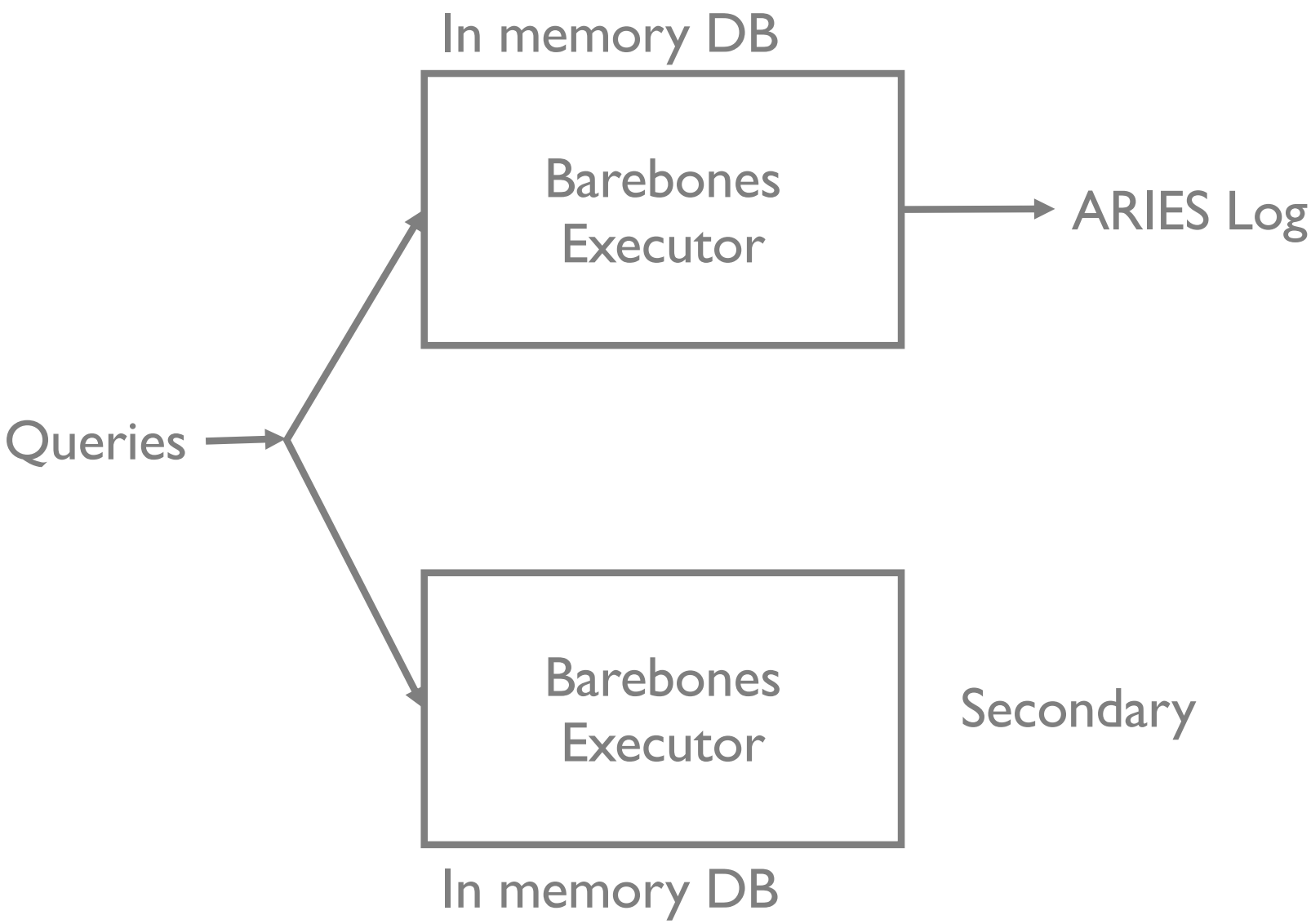
Log Shipping



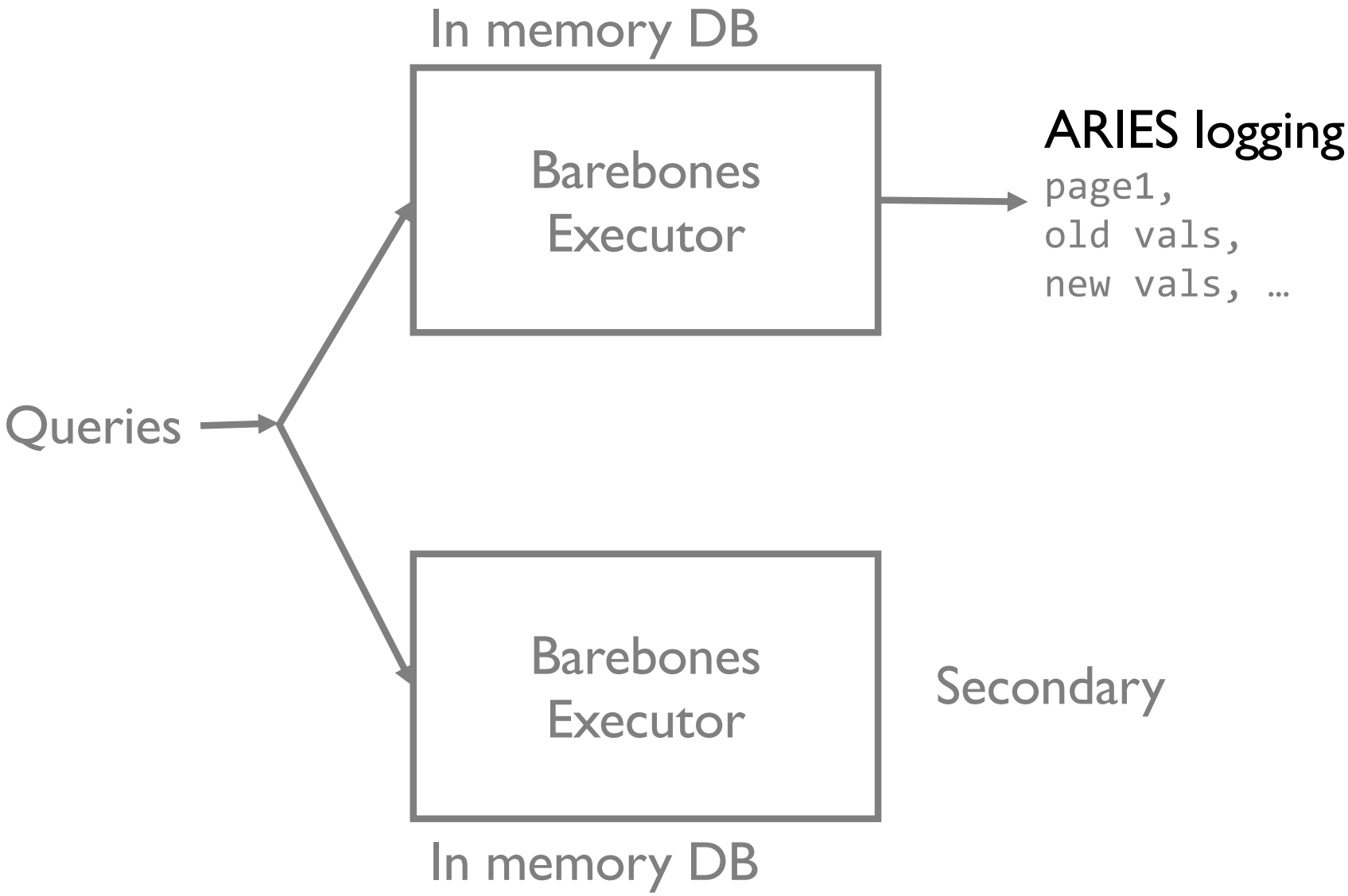
Active-Active



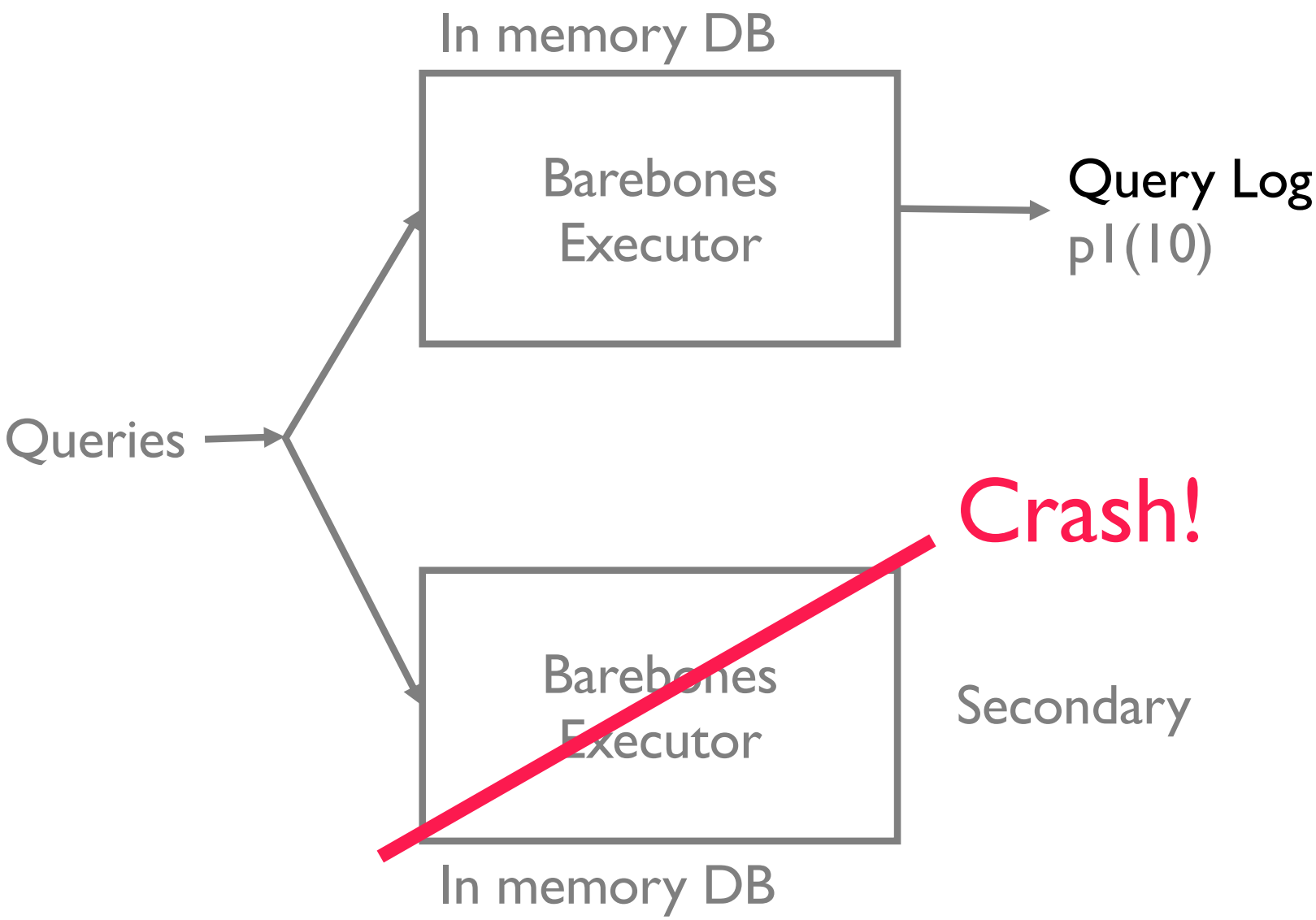
Recovery in Active-Active



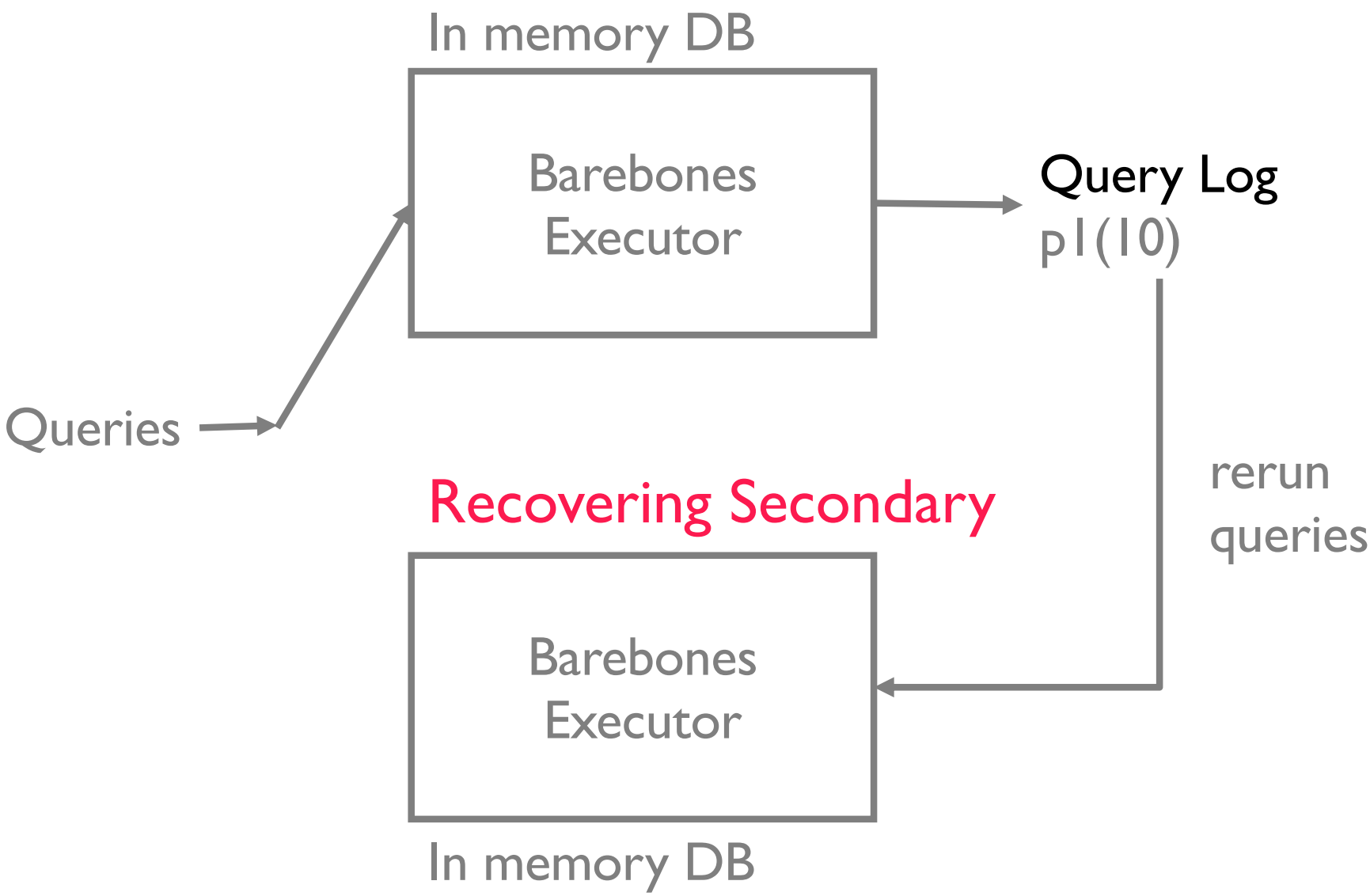
Recovery in Active-Active



Recovery in Active-Active

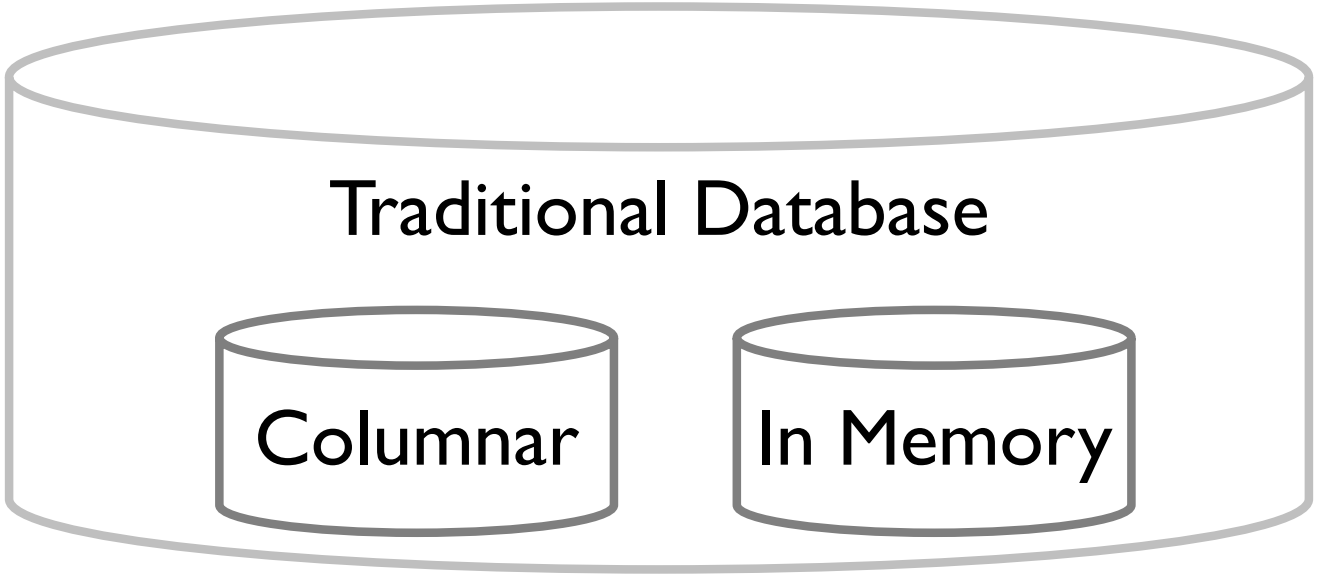


Recovery in Active-Active

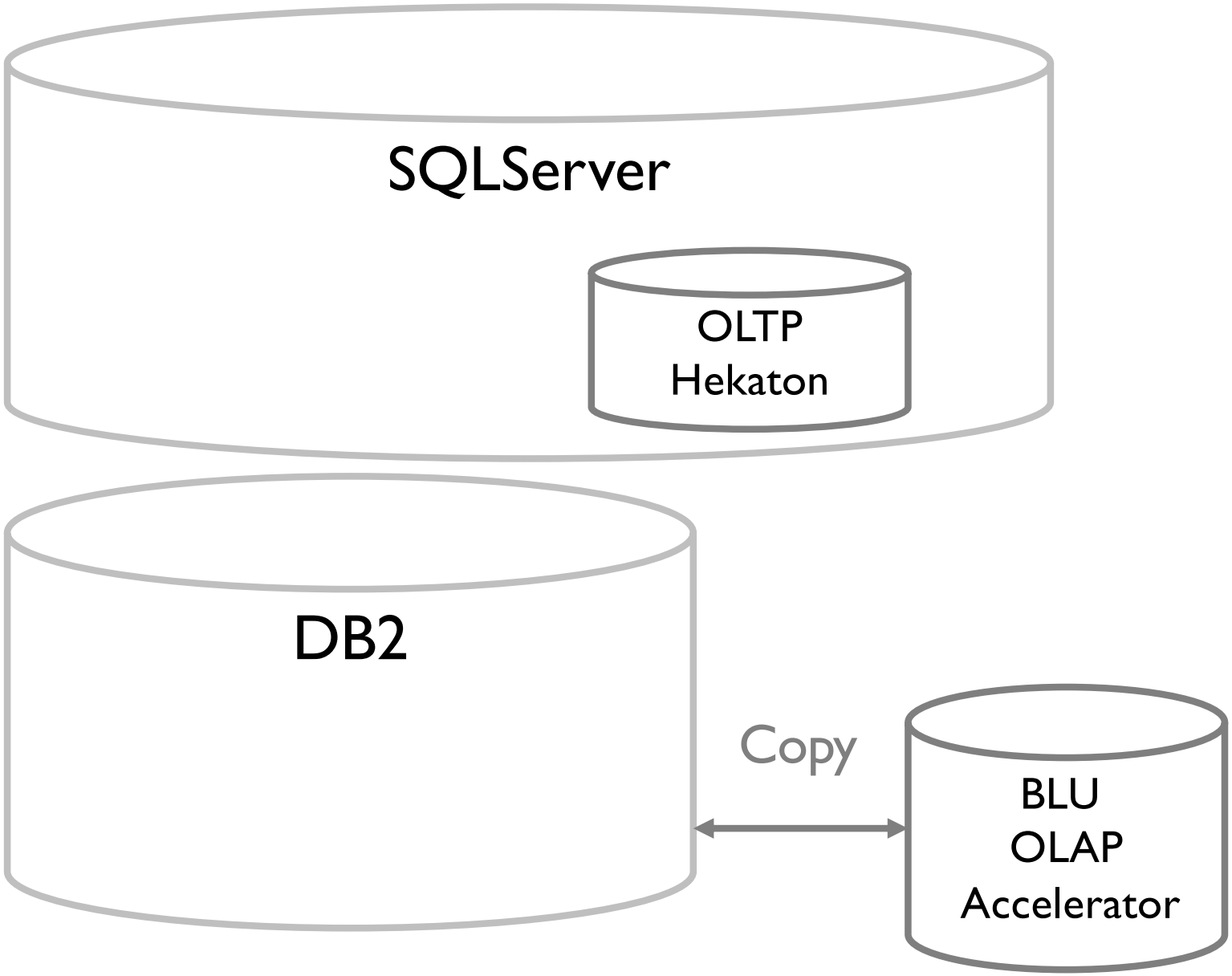


One Size Does Not Fits All

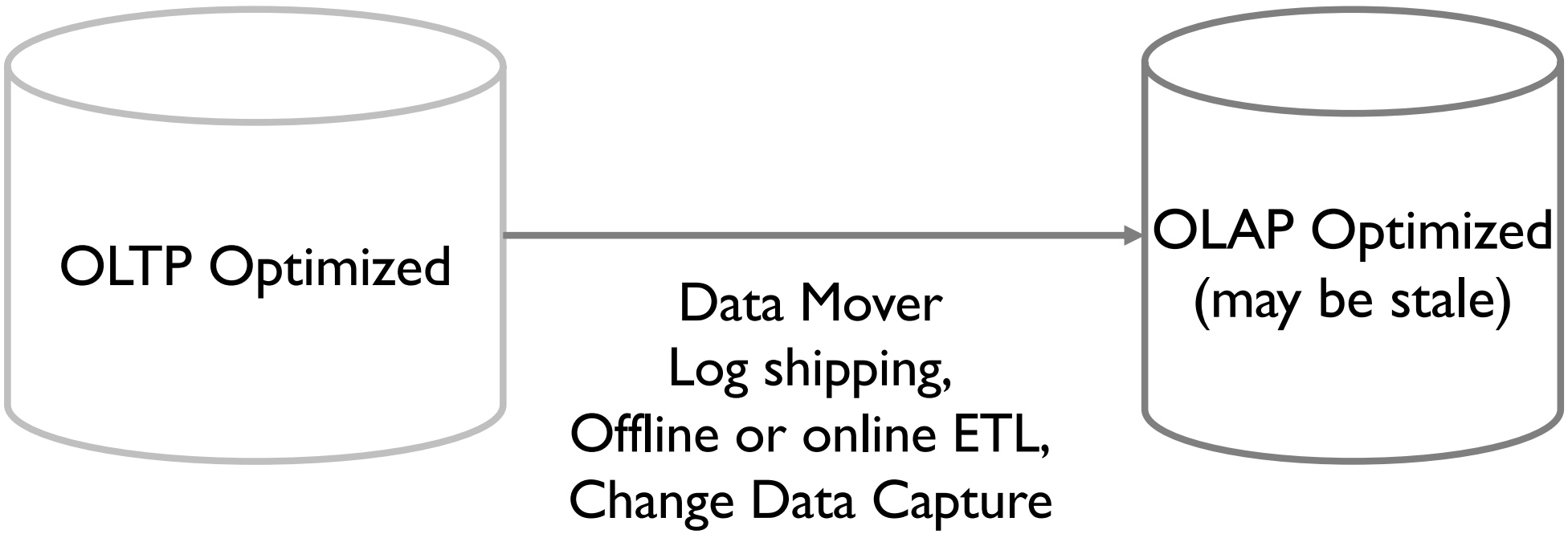
One Size Does Not Fits All



One Size Does Not Fits All

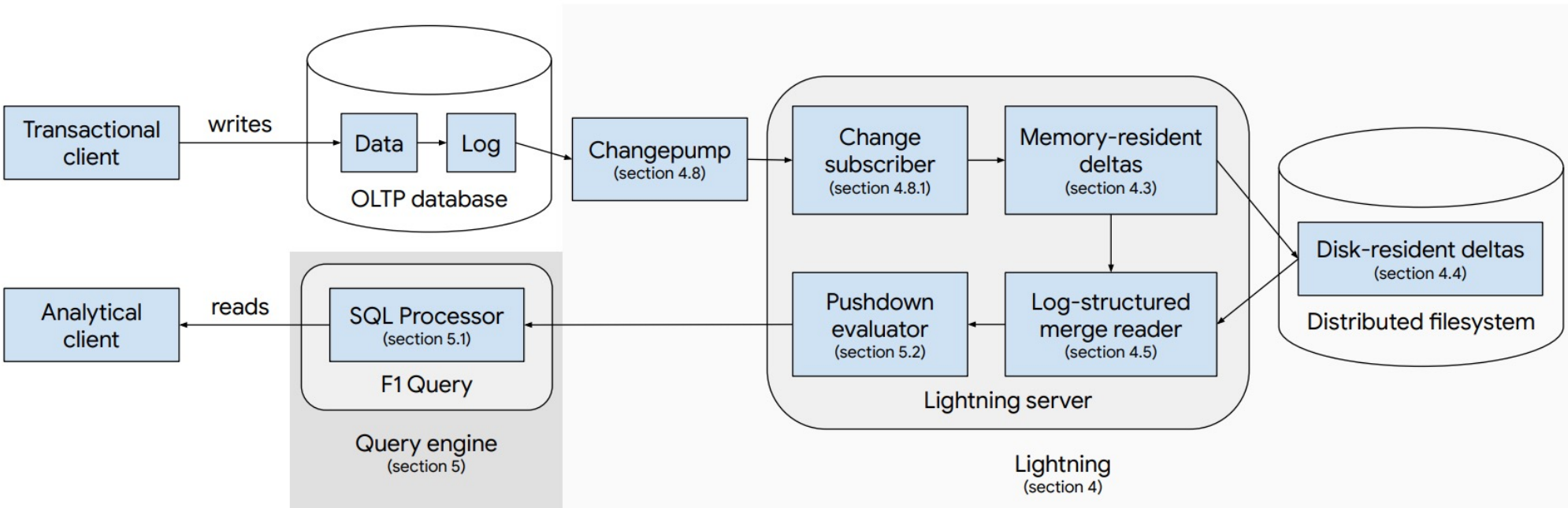


“HTAP”


OLTP Optimized
Data Mover
Log shipping,
Offline or online ETL,
Change Data Capture
OLAP Optimized
(may be stale)

F1 Lightning: HTAP as a Service

Jiacheng Yang Ian Rae Jun Xu Jeff Shute Zhan Yuan Kelvin Lau
Qiang Zeng Xi Zhao Jun Ma Ziyang Chen Yuan Gao Qilin Dong
Junxiong Zhou Jeremy Wood Goetz Graefe Jeff Naughton John Cieslewicz
Google LLC
f1-lightning-paper@google.com

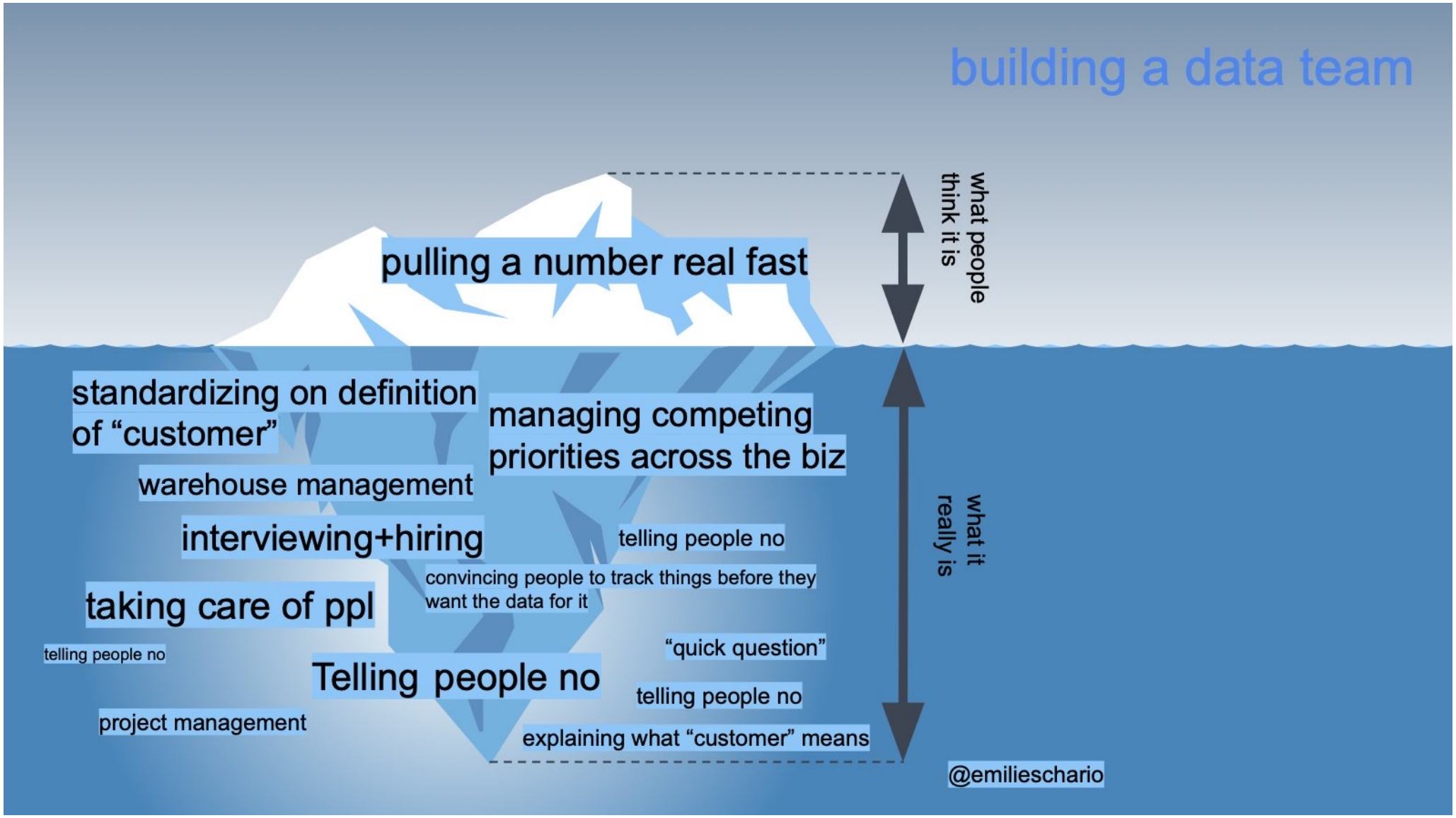


OK Let's Step Back

Discussed the *how*:

- how to model data needed by an application
- how to query databases
- how databases execute queries
- how databases run fast, correctly

To what end?



https://twitter.com/emilieschario/status/1427419697654419456/p

Production ML

An on-call engineer's biggest nightmare

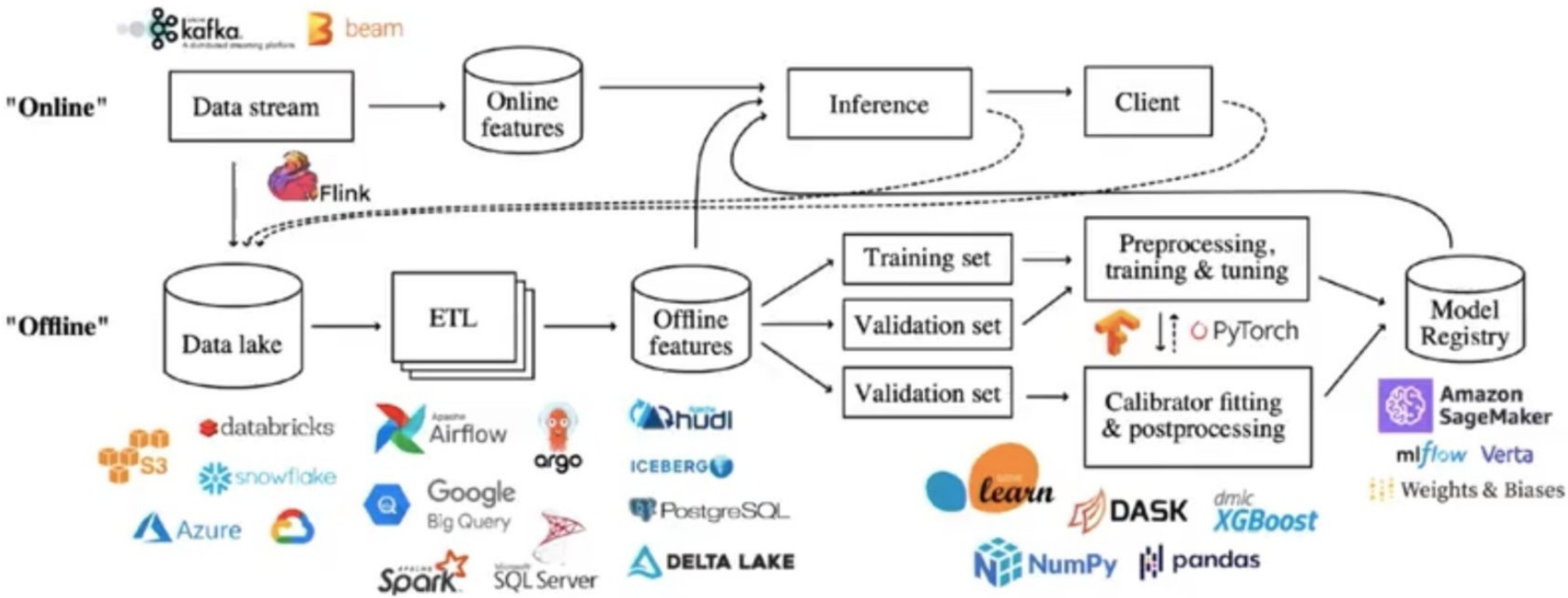
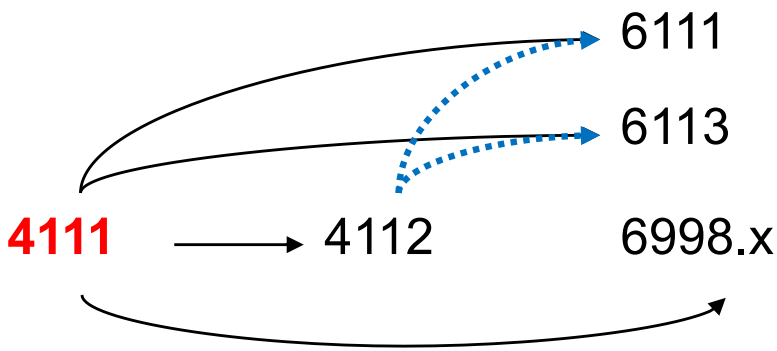


Figure 1: High-level architecture of a generic end-to-end machine learning pipeline. Logos represent a sample of tools used to construct components of the pipeline, illustrating heterogeneity in the tool stack. *Shankar et al. 2021*

https://www.facebook.com/Engineering/videos/1578607659138164/



4xxx: "here are facts about data management"

4111: basics

4112: gory DBMS internals

6xxx: "how people figured out those facts"

6111: info extraction and web

6113: class and modern DB research

6998: systems for human data interaction

PhD: "we don't understand anything. Plz help"

Thanks!

Please fill out courseworks survey

cool.